



دانشگاه شهید بهشتی
دانشکده مهندسی و علوم کامپیوتر

استدلال در مدل های زبانی کوچک برای دامنه حل مسائل ریاضی

پروژه کارشناسی
رشته مهندسی کامپیوتر

دانشجو:
پوریا یزدانی

استاد راهنما:
دکتر حامد ملک

زمستان ۱۴۰۴

چکیده

با وجود پیشرفت چشمگیر مدل‌های زبانی بزرگ، بهبود توان استدلال در مدل‌های زبانی کوچک همچنان یکی از چالش‌های مهم در کاربردهای کم‌هزینه و قابل‌استقرار محلی است. این پروژه با هدف تقویت استدلال در یک مدل زبانی کوچک، تحت محدودیت‌های عملی مانند حافظه، توان محاسباتی و سخت‌افزار شخصی، طراحی و اجرا شده است. در گام نخست، مفاهیم و روش‌های موجود برای بهبود استدلال در مدل‌های زبانی، از جمله مقیاس‌دهی در زمان استنتاج، روش‌های مبتنی بر برنامه‌ریزی، خوداعتبارسنجی و تنظیم سبک‌وزن، بررسی شدند. بر مبنای این مطالعه، یک معماری دو مرحله‌ای انتخاب شد که در آن مدل ابتدا برنامه حل را تولید می‌کند و سپس بر پایه همان برنامه به حل مسئله می‌پردازد. برای ارزیابی و بهبود این ایده، دامنه حل مسئله ریاضی و مدل Qwen3-1.7B انتخاب شد. سپس تحلیل خطا بر خروجی‌های مرحله برنامه‌ریزی انجام گرفت و مسائل ریاضی بر اساس الگوهای تکرارشونده خطا خوشه‌بندی شدند. در ادامه، خوشه مسائل عددی بلند و چندقیدی، به‌عنوان خوشه دارای خطاهای غالب و بحرانی، برای مداخله انتخاب شد. برای این خوشه، رفتار برنامه‌ریزی انسان‌گونه تعریف شد. رفتاری که بر استخراج هدف، مجهول‌ها، قیود مؤثر و اسکلت وابستگی پیش از حل تأکید دارد. سپس یک مجموعه داده شامل ۵۰ خانواده مسئله و ۵۰۰ نمونه از روی این خوشه تولید شد و با استفاده از یک مدل معلم، برای هر نمونه برنامه حل ساختاریافته تولید گردید. در نهایت، مدل کوچک با استفاده از تنظیم دقیق با نظارت سبک‌وزن مبتنی بر QLoRA آموزش داده شد. نتایج نشان داد که مدل تنظیم‌شده برنامه‌های حل منسجم‌تر و دقیق‌تری تولید می‌کند و این بهبود در کیفیت برنامه‌ریزی، به افزایش تعداد مسئله‌های حل‌شده به‌درستی منجر شده است. این یافته‌ها نشان می‌دهد که معماری پیشنهادی ظرفیت مناسبی برای بهبود استدلال در دامنه ریاضی دارد و می‌تواند در آینده برای سایر خوشه‌های مسئله نیز گسترش یابد.

واژگان کلیدی: مدل‌های زبانی کوچک، استدلال، برنامه‌ریزی و حل، حل مسئله ریاضی، تنظیم دقیق با نظارت سبک‌وزن، تحلیل خطا

فهرست مطالب

1	فصل اول: کلیات
2	۱-۱ مقدمه
3	۲-۱ بیان مسئله
3	۱-۲-۱ استدلال در مدل‌های زبانی
4	۲-۲-۱ محدودیت استدلال در مدل‌های زبانی کوچک
5	۳-۲-۱ محدودیت‌های عملی پروژه
5	۴-۲-۱ اهداف، پرسش‌ها و دامنه پروژه
6	۳-۱ کلیات روش پیشنهادی
6	۱-۳-۱ مرور ادبیات و شناسایی مسیرهای ممکن
7	۲-۳-۱ انتخاب معماری دو مرحله‌ای برنامه‌ریزی و حل
8	۳-۳-۱ مسیر کلی محدودسازی و بهبود مدل
9	۴-۱ ساختار پروژه
10	فصل دوم: مفاهیم پایه و کارهای مرتبط
11	۱-۲ مقدمه
11	۲-۲ مفاهیم پایه
11	۱-۲-۲ استدلال در مدل‌های زبانی
12	۲-۲-۲ استدلال در حل مسئله ریاضی
13	۳-۲-۲ ساختار میان‌مرحله‌ای در فرایند استدلال
14	۴-۲-۲ برنامه‌ریزی، تجزیه مسئله و حل مرحله‌ای
15	۳-۲ روش‌های بهبود استدلال در استنتاج
17	۴-۲ کارهای مرتبط
25	فصل سوم: روش پیشنهادی نتیجه‌گیری
26	۱-۳ مقدمه
26	۲-۳ مدل، دامنه مسائل و معماری پیشنهادی

26	۱-۲-۳ انتخاب مدل و دامنه مسئله
28	۲-۲-۳ معماری پیشنهادی
31	۳-۳ تحلیل خطا روی مسائل دامنه ریاضی
31	۱-۳-۳ تحلیل اولیه خروجی‌های مرحله برنامه‌ریزی
33	۲-۳-۳ خوشه‌بندی مسائل
37	۱-۴-۳ تعریف رفتار برنامه‌ریزی انسان‌گونه
40	۲-۴-۳ تولید داده با مدل معلم
43	۱-۴-۳ تنظیم دقیق سبک‌وزن مدل با LoRA
44	۲-۴-۳ ابزارها، محیط اجرا
45	۶-۳ نتایج و ارزیابی
46	۷-۳ جمع‌بندی و کارهای آینده

فهرست شکل‌ها

- شکل 1-1 چارچوب مطالعه تقسیم‌بندی روش‌های استدلال در این پروژه 7
- شکل 1-2 پرامپت مبتنی بر CoT 18
- شکل 2-2 پرامپت‌های Zero-shot, Few-shot, CoT و Zero-shot-CoT 19
- شکل 2-3 پرامپت مبتنی بر Plan-and-Solve 19
- شکل 3-1 خط لوله روش پیشنهادی 29
- شکل 3-2 نحوه استنتاج در معماری پیشنهادی 30
- شکل 3-3 پرامپت استخراج برنامه حل گام به گام 39

فصل اول: کلیات

۱-۱ مقدمه

پیشرفت مدل‌های زبانی بزرگ^۱ در سال‌های اخیر، به‌ویژه پس از تثبیت معماری ترنسفورمر^۲ و گسترش آموزش بر مجموعه داده‌های بسیار بزرگ، باعث شده است این مدل‌ها در طیف وسیعی از وظایف زبانی و تا حدی در وظایف استدلالی^۳ عملکردی چشمگیر از خود نشان دهند. با این حال، موفقیت در تولید متن روان یا پاسخ‌های ظاهراً قانع‌کننده، الزاماً به معنای برخورداری از استدلال دقیق، قابل‌اعتماد و مرحله‌مند نیست. در بسیاری از مسائل پیچیده، به‌خصوص مسائلی که نیازمند حفظ قیود، کنترل وابستگی‌ها و حرکت نظام‌مند از داده‌های مسئله به پاسخ نهایی هستند، کیفیت فرایند استدلال همچنان به‌عنوان یکی از چالش‌های اصلی مدل‌های زبانی باقی مانده است. [۱]، [۲]، [۳]، [۴]

اهمیت این موضوع زمانی بیشتر می‌شود که به‌جای استفاده از مدل‌های بسیار بزرگ، ناچار به کار با مدل‌های زبانی کوچک^۴ باشیم. این دسته از مدل‌ها از نظر هزینه اجرا، حافظه موردنیاز و امکان استقرار محلی بسیار مورد توجه هستند، اما در وظایف چندمرحله‌ای و در سناریوهایی که خطاهای کوچک می‌توانند کل مسیر پاسخ را منحرف کنند، با ضعف‌های جدی‌تری روبه‌رو می‌شوند. از این‌رو، مسئله اصلی این پروژه نه ساخت یک مدل بزرگ‌تر، بلکه یافتن راهی برای تقویت رفتار استدلالی در یک مدل کوچک، آن هم تحت محدودیت‌های واقعی سخت‌افزاری و محاسباتی، بوده است.

بر همین اساس، مسیر پروژه از یک مطالعه گسترده و بدون دامنه مشخص آغاز شد؛ یعنی در ابتدا تمرکز روی یک حوزه خاص مانند ریاضیات نبود، بلکه پرسش اصلی این بود که چه روش‌هایی برای تقویت استدلال در مدل‌های زبانی وجود دارند، کدامیک با محدودیت‌های عملی این پروژه سازگارترند، و چگونه می‌توان از میان آن‌ها به یک طراحی اجرایی و قابل‌پیاده‌سازی رسید. نتیجه این مرحله، رسیدن به این برداشت بود که برای مدل‌های کوچک، کنترل ساختار فرایند استدلال در زمان استنتاج، مسیر واقع‌بینانه‌تری نسبت به تکیه صرف بر افزایش اندازه مدل یا بازآموزی‌های بسیار پرهزینه است.

¹ Large Language Model (LLM)

² Transformer Architecture

³ Reasoning Tasks

⁴ Small Language Model (SLM)

فصل اول: کلیات

در ادامه این فصل، ابتدا مسئله پژوهش تعریف می‌شود، سپس انگیزه انتخاب رویکرد پیشنهادی توضیح داده می‌شود، و در پایان ساختار کلی گزارش تشریح خواهد شد. هدف این فصل آن است که خواننده را از سطح کلی مسئله استدلال در مدل‌های زبانی به منطق انتخاب روش این پروژه برساند؛ به گونه‌ای که مسیر شکل‌گیری ایده اصلی پروژه به صورت طبیعی و منسجم روشن شود.

۲-۱ بیان مسئله

۱-۲-۱ استدلال در مدل‌های زبانی

استدلال^۵ به فرایندی گفته می‌شود که در آن مدل بتواند از داده‌های مسئله به صورت ساختارمند عبور کند، وضعیت‌های میانی را به شکل معنادار بسازد، و میان قیود، متغیرها و نتیجه نهایی رابطه‌ای منظم برقرار کند. ادبیات پژوهش در سال‌های اخیر نشان داده است که با استفاده از روش‌هایی مانند Chain-of-Thought Prompting، می‌توان مدل را وادار کرد که به جای پاسخ مستقیم، مسیر میانی^۶ خود را نیز آشکار کند و همین امر در بسیاری از وظایف استدلالی موجب بهبود عملکرد می‌شود. [۵]

پس از آن، Zero-shot-CoT نشان داد که حتی بدون ارائه نمونه‌های آماده، افزودن یک پرامپت^۷ ساده برای فکر کردن گام به گام نیز می‌تواند کیفیت پاسخ را در بخشی از مسائل استدلالی بالا ببرد. در ادامه، مفهوم Self-Consistency این ایده را یک گام جلوتر برد و به جای تکیه بر یک مسیر واحد، از چند مسیر استدلالی استفاده کرد و پاسخی را برگزید که سازگاری بیشتری با سایر مسیرها داشت. این روند نشان می‌دهد که استدلال در مدل‌های زبانی بیش از آنکه صرفاً وابسته به وزن‌های مدل باشد، به نحوه سازمان‌دهی فرایند تولید نیز متن وابسته است. [۶]، [۷]

⁵ Reasoning

⁶ Intermediate Steps

⁷ Prompt

با این حال، آشکارسازی مسیر میانی به‌تنهایی برای حل مسئله کافی نیست. در بسیاری از موارد، مدل می‌تواند زنجیره‌ای^۸ از گام‌ها تولید کند که از نظر نگارشی منسجم‌اند اما از منظر منطقی یا ساختاری دچار نقص هستند؛ برای مثال، ممکن است یک قید مهم را نادیده بگیرد، وابستگی میان دو بخش مسئله را اشتباه مدل کند، یا ترتیب لازم برای رسیدن به پاسخ را حفظ نکند. بنابراین، مسئله اصلی در این پروژه نه فقط تولید زنجیره‌ای از متن، بلکه هدایت مدل به سوی ساخت یک اسکلت استدلالی منظم و کم‌خطا بوده است. [۸]، [۱۰]، [۱۱]

۱-۲-۲ محدودیت استدلال در مدل‌های زبانی کوچک

قوانین مقیاس‌پذیری^۹ در مدل‌های زبانی نشان می‌دهند که با افزایش اندازه مدل، داده آموزشی و بودجه محاسباتی، کیفیت مدل در بسیاری از وظایف به‌صورت پیش‌بینی‌پذیر بهبود می‌یابد. همچنین پژوهش‌های مربوط به آموزش بهینه از نظر محاسباتی نیز نشان داده‌اند که نسبت میان اندازه مدل و تعداد توکن‌های^{۱۰} آموزشی نقش مهمی در کیفیت نهایی دارد. این نتیجه به‌طور طبیعی توضیح می‌دهد که چرا بسیاری از رفتارهای استدلالی قوی‌تر ابتدا در مدل‌های بسیار بزرگ ظاهر شده‌اند. اما در عمل، استفاده از چنین مدل‌هایی همیشه ممکن نیست. برای بسیاری از سامانه‌هایی که باید روی سخت‌افزار شخصی یا محیط‌های محدود اجرا شوند، هزینه محاسباتی، نیاز حافظه و دشواری استقرار، استفاده از مدل‌های بسیار بزرگ را به گزینه‌ای غیرعملی تبدیل می‌کند. در این وضعیت، مدل‌های کوچک از نظر اجرایی انتخاب مناسبی‌اند، اما معمولاً در استدلال چندمرحله‌ای، حفظ انسجام مسیر حل، و جلوگیری از پرش‌های نادرست در زنجیره تولید ضعف بیشتری دارند. [۵]، [۶]، [۱۳]

از این‌جا مسئله اصلی پروژه شکل می‌گیرد: آیا می‌توان بدون بزرگ‌تر کردن مدل، و در همان چارچوب محدود منابع، کیفیت استدلال را بالا برد؟ پاسخ اولیه ادبیات این است که بخشی از این بهبود می‌تواند نه از راه تغییر کامل خود مدل، بلکه از راه تغییر در نحوه استفاده از مدل حاصل شود؛ یعنی با طراحی بهتر فرایند استنتاج، تولید چند مسیر، بررسی و پالایش آن‌ها، یا ساخت خروجی‌های میانی ساختاریافته‌تر. [۷]، [۹]، [۱۰]، [۱۱]

^۸ Chain

^۹ Scalability

^{۱۰} Token

۱-۲-۳ محدودیت‌های عملی پروژه

این پروژه از ابتدا تحت مجموعه‌ای از محدودیت‌های عملی تعریف شد که بر انتخاب روش‌ها اثر مستقیم داشتند. محدودیت توان محاسباتی، محدودیت حافظه، دشواری اجرای مدل‌های بزرگ روی سخت‌افزار محلی، و هزینه زمانی و فنی تنظیم و اجرای مدل‌ها، همگی باعث شدند بخش قابل‌توجهی از راه‌حل‌های سنگین از دایره انتخاب خارج شوند. به همین دلیل، این پروژه از همان ابتدا به دنبال رویکردهایی بود که در عین حفظ ارزش پژوهشی، از نظر اجرا نیز واقع‌بینانه باشند.

این محدودیت‌ها یک پیامد روش‌شناختی مهم داشتند. اگر قرار بود استدلال در مدل‌های کوچک تقویت شود، راه‌حل باید یا در زمان استنتاج هزینه‌ای قابل‌کنترل داشته باشد، یا از تنظیم سبک‌وزن استفاده کند، یا هر دو. به بیان دیگر، مسیرهای مبتنی بر مقیاس‌دهی در زمان استنتاج^{۱۱}، نمونه‌گیری کنترل‌شده، داوری مرحله‌ای، و تنظیم دقیق^{۱۲} بهینه از نظر پارامتری مانند LORA (۱۰) به‌طور طبیعی در مرکز توجه قرار گرفتند، در حالی که روش‌هایی که به آموزش‌های سنگین و تمام‌پارامتری نیاز دارند، برای این پروژه عملی نبودند.

در نتیجه، پروژه به جای دنبال کردن هم‌زمان همه راه‌های موجود، ابتدا بر مطالعه مفهومی و شناسایی دسته‌های روش‌های استدلال تمرکز کرد، سپس از میان آن‌ها یک مسیر مشخص و سازگار با محدودیت‌ها را انتخاب شد. همین منطق موجب شد که به‌جای شروع مستقیم از یک دامنه مشخص، روند پژوهشی ابتدا مسئله را در سطح عام استدلال در مدل‌های زبانی بررسی کند و در مراحل بعدی آن را به یک مدل، یک دامنه و سپس یک نوع مسئله محدود سازد.

۱-۲-۴ اهداف، پرسش‌ها و دامنه پروژه

هدف اصلی این پروژه، تقویت توان استدلال در مدل‌های زبانی کوچک از طریق طراحی و اعمال یک فرایند استدلالی ساختاریافته و قابل‌کنترل بوده است. این هدف بر این فرض استوار است که ضعف بسیاری از مدل‌های کوچک فقط از کمبود دانش یا کوچکی اندازه ناشی نمی‌شود، بلکه بخشی از آن به نبود سازمان‌دهی مناسب در فرایند استنتاج مربوط است؛ بنابراین،

¹¹ Inference-Time Scaling

¹² Fine-tune

فصل اول: کلیات

اگر بتوان مرحله‌ای از استدلال را که نقش سازمان‌دهنده دارد به‌خوبی تعریف و تقویت کرد، می‌توان انتظار داشت کیفیت خروجی نهایی نیز بهتر شود. [۹]، [۱۰]، [۱۱]

بر این اساس، پرسش‌های اصلی پروژه را می‌توان چنین صورت‌بندی کرد:

نخست: چه دسته‌ای از روش‌های موجود برای تقویت استدلال با محدودیت‌های عملی این پروژه سازگارترند؟

دوم: آیا جدا کردن مرحله ساخت برنامه حل مسئله از مرحله اجرای حل مسئله می‌تواند به‌عنوان یک نقطه مداخله مؤثر برای بهبود رفتار استدلالی عمل کند؟

سوم: آیا می‌توان بخشی از الگوی استدلال ساختاریافته یک مدل قوی را به یک مدل کوچک منتقل کرد، به‌طوری‌که این مدل در مواجهه با مسائل چندمرحله‌ای مسیر حل منظم‌تری تولید کند؟

دامنه این فصل، تعریف مسئله و ترسیم جهت‌گیری علمی پروژه است و هدف آن این است که نشان دهد چرا از میان مجموعه‌ای از مسیرهای ممکن، یک رویکرد مشخص برای ادامه پروژه مناسب‌تر تشخیص داده شد.

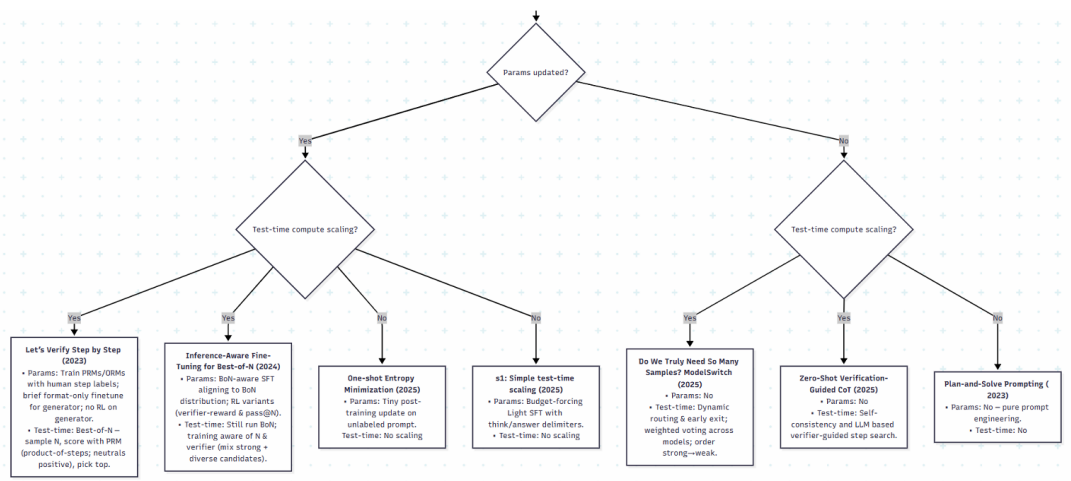
۳-۱ کلیات روش پیشنهادی

۳-۱-۱ مرور ادبیات و شناسایی مسیرهای ممکن

مرحله نخست پروژه با مطالعه‌ای گسترده بر مفاهیم و مقالات مرتبط با استدلال در مدل‌های زبانی آغاز شد. این مرور بر شناسایی خانواده‌های اصلی روش‌های تقویت استدلال با توجه به دامنه پروژه و در نظر گرفتن محدودیت توان محاسباتی متمرکز بود در نتیجه روش‌های مبتنی بر یادگیری تقویتی و مدل‌های پاداش پس از مطالعه اولیه از دامنه کاری این پروژه خارج شدند. برخی از این روش‌ها در [10][22] بررسی شده‌اند. از دل این مطالعه، چند محور اصلی هم‌راستا با محدودیت‌های مربوط به توان محاسباتی پروژه استخراج شد: زنجیره فکر و انواع پرامپت‌های استدلالی، مقیاس‌دهی در زمان استنتاج، نمونه‌گیری و انتخاب از میان چند مسیر، داوری و خوداعتبارسنجی، پالایش تدریجی، تجزیه مسئله به گام‌های اتمی، و روش‌های تنظیم کارآمد برای انتقال رفتارهای استدلالی به مدل‌های کوچک. این بررسی اساسی یک نتیجه مهم داشت: همه روش‌های بهبود استدلال در یک سطح قرار نمی‌گیرند. بعضی روش‌ها صرفاً نحوه دستور دادن به مدل را تغییر می‌دهند، بعضی

فصل اول: کلیات

در زمان استنتاج محاسبه بیشتری صرف می‌کنند، بعضی از یک داور یا مدل پاداش برای انتخاب مسیر بهتر استفاده می‌کنند، و بعضی دیگر سعی می‌کنند ساختار استدلال را از راه تنظیم مدل پایدارتر کنند. بنابراین، پیش از طراحی هر سامانه عملی، لازم بود این مسیرها از منظر سازگاری با محدودیت‌های پروژه و از منظر قابلیت مداخله در کیفیت استدلال، مقایسه و غربال شوند. [۵]، [۶]، [۹]، [۱۰]، [۹]، [۱۰]، [۱۱]، [۱۲].



شکل 1-1 چارچوب مطالعه تقسیم‌بندی روش‌های استدلال در این پروژه

۲-۳-۱ انتخاب معماری دو مرحله‌ای برنامه‌ریزی و حل

پس از مقایسه مسیرهای مختلف، ایده طراحی شده، معماری دو مرحله‌ای برنامه‌ریزی و حل^{۱۳} بود. منطق این انتخاب آن بود که در بسیاری از مسائل دشوار، شکست مدل نه در خود محاسبه نهایی، بلکه در ناتوانی از ساختن یک برنامه حل منظم رخ می‌دهد. اگر مدل نتواند هدف مسئله، مجهولات واقعی، قیود اصلی و وابستگی‌های شرطی را درست تشخیص دهد، حتی با توان محاسباتی بیشتر نیز احتمالاً به پاسخ درست نخواهد رسید. Plan-and-Solve Prompting دقیقاً بر همین نکته تکیه می‌کند و نشان می‌دهد که افزودن یک مرحله مستقل برای طرح‌ریزی پیش از حل می‌تواند بخشی از خطاهای ناشی از جافتادن گام‌ها و سوءبرداشت ساختاری را کاهش دهد [۹].

¹³ Plan-and-Solve

فصل اول: کلیات

در این پروژه، برداشت از مرحله برنامه‌ریزی از همان ابتدا محدود به یک زنجیره فکر عمومی نبود. منظور از برنامه‌ریزی، تولید یک اسکلت حداقلی و انسان‌گونه برای حل بود؛ اسکلتی که در آن هدف مسئله مشخص شود، فقط مجهولات واقعی فهرست شوند، قیود مؤثر استخراج شوند، و رابطه‌های وابستگی و شاخه‌های شرطی به‌صورت روشن بازنمایی شوند، اما هنوز هیچ محاسبه یا حل عددی انجام نشود. به این ترتیب، مرحله نخست نقش سازمان‌دهنده فرایند حل را بر عهده می‌گیرد و مرحله دوم تنها پس از دریافت این ساختار وارد اجرای حل می‌شود. این انتخاب از منظر روش‌شناختی نیز مزیت مهمی داشت. وقتی مرحله برنامه‌ریزی از مرحله حل جدا می‌شود، می‌توان کیفیت برنامه را جداگانه تحلیل کرد، خطاهای آن را دسته‌بندی نمود، و حتی در صورت لزوم فقط همان مرحله را هدف تنظیم یا آموزش قرار داد. این جداسازی برای پروژه‌ای که هم محدودیت محاسباتی داشت و هم به دنبال یک نقطه مداخله مشخص و قابل تحلیل بود، بسیار تعیین‌کننده بود. [۸]، [۱۰]، [۱۱]

۱-۳-۳ مسیر کلی محدودسازی و بهبود مدل

پس از انتخاب این معماری، مسیر پروژه به‌تدریج محدودتر و عملی‌تر شد. منطق کلی این مسیر چنین شکل گرفت: ابتدا باید مشخص شود که در چه مرحله‌ای از این معماری امکان مداخله مؤثرتری وجود دارد؛ سپس همان مرحله به‌صورت مستقل تحلیل شود و در نهایت با داده و تنظیم مناسب، همان بخش بهبود یابد. این منطق به‌صورت طبیعی موجب شد که تمرکز اصلی ابتدا بر مرحله برنامه‌ریزی قرار گیرد، نه بر کل فرایند حل. [۹]، [۱۰]، [۱۲] از منظر فنی نیز این انتخاب منطقی بود. اگر قرار باشد یک مدل کوچک با هزینه کمتر بهبود یابد، تنظیم کل فرایند پاسخ‌گویی در بسیاری از موارد پرهزینه‌تر و مبهم‌تر از تنظیم یک مرحله مشخص است.

بنابراین، کلیات روش پیشنهادی در این پروژه بر سه تصمیم کلیدی استوار شد: نخست، درک مسئله تقویت استدلال در سطح عمومی. دوم، انتخاب یک معماری دو مرحله‌ای که ساختار استدلال را از اجرای حل جدا کند؛ و سوم، تبدیل مرحله برنامه‌ریزی به نقطه اصلی مداخله برای تحلیل و بهبود بعدی. این سه تصمیم، چارچوب مفهومی فصل‌های بعدی پروژه را شکل داده‌اند.

۴-۱ ساختار پروژه

گزارش حاضر در سه فصل اصلی تنظیم شده است. فصل نخست به تعریف مسئله، بیان انگیزه‌ها و تشریح کلیات روش پیشنهادی اختصاص دارد. در این فصل تلاش شده است زمینه نظری پروژه و منطق شکل‌گیری ایده اصلی آن به صورت منسجم روشن شود. فصل دوم به مفاهیم پایه و کارهای مرتبط می‌پردازد. در این فصل، مفاهیم اصلی مورد نیاز برای فهم پروژه، از جمله استدلال در مدل‌های زبانی، ساختارهای میان‌مرحله‌ای، برنامه‌ریزی و روش‌های بهبود استدلال در زمان استنتاج بررسی می‌شود و سپس مهم‌ترین پژوهش‌های مرتبط مرور و تحلیل می‌گردد. در فصل سوم، روش پیشنهادی در سطح اجرایی تشریح خواهد شد؛ شامل انتخاب مدل و دامنه، تحلیل خطا، ساخت داده، تنظیم دقیق مدل، ارزیابی و نتایج. به این ترتیب، دو فصل نخست نقش زمینه‌سازی مفهومی و روش‌شناختی را ایفا می‌کنند و فصل سوم بخش عملی و پیاده‌سازی پروژه را کامل می‌کند.

فصل دوم: مفاهیم پایه و کارهای مرتبط

۱-۲ مقدمه

فصل حاضر نقش پیونددهنده میان تعریف کلی مسئله در فصل نخست و طراحی عملی روش در فصل بعدی را بر عهده دارد. در این فصل، ابتدا مفاهیمی تشریح می‌شوند که برای فهم دقیق مسئله و انتخاب مسیر مناسب پژوهش ضروری‌اند؛ سپس خانواده‌های اصلی روش‌های بهبود استدلال بررسی می‌شوند و در پایان، مهم‌ترین کارهای مرتبطی که در شکل‌گیری مسیر این پروژه اثر داشته‌اند مرور و تحلیل می‌گردند. این فصل عمده‌اً در سطح مفهومی و روش‌شناختی باقی می‌ماند، زیرا هدف آن پیش از ورود به جزئیات اجرا، روشن کردن این نکته است که مسئله استدلال در مدل‌های زبانی چگونه فهم شده، چه راه‌حلهایی برای آن پیشنهاد شده، و چرا برخی از این راه‌حل‌ها برای پروژه حاضر مناسب‌تر از دیگر گزینه‌ها بوده‌اند. از منظر مفهومی، این فصل بر چهار پرسش اصلی بنا شده است: استدلال در مدل‌های زبانی دقیقاً به چه معناست؛ چرا حل مسئله ریاضی بستری مناسب برای مطالعه استدلال است؛ ساختارهای میان‌مرحله‌ای چه نقشی در کاهش خطا دارند؛ و چگونه می‌توان با مداخله در زمان استنتاج یا از طریق تنظیم سبک‌وزن، عملکرد استدلالی یک مدل را بهبود داد. پاسخ به این پرسش‌ها در کنار مرور کارهای مرتبط، پایه‌ای فراهم می‌کند که بر مبنای آن انتخاب معماری دو مرحله‌ای برنامه‌ریزی و حل در فصل بعدی طبیعی و موجه به نظر برسد.

۲-۲ مفاهیم پایه

۱-۲-۲ استدلال در مدل‌های زبانی

استدلال در مدل‌های به توانایی مدل در ساخت و پیمایش یک مسیر میانی اشاره دارد که از ورودی مسئله آغاز می‌شود، از چند گام یا حالت واسطه عبور می‌کند، و در نهایت به یک نتیجه قابل قبول می‌رسد. اهمیت این تعریف در آن است که بسیاری از پاسخ‌های درست می‌توانند حاصل حدس، میان‌بر آماری یا تکرار الگوهای پرتکرار باشند، در حالی که در مسائل پیچیده، آنچه اهمیت دارد نه فقط خود پاسخ، بلکه کیفیت سازمان‌دهی مسیر رسیدن به پاسخ است. پژوهش Chain-of-Thought Prompting نشان داد که واداشتن مدل به تولید گام‌های میانی می‌تواند عملکرد آن را در مسائل حسابی، نمادین و استدلال عمومی بهبود دهد، و پژوهش Large Language Models are Zero-Shot Reasoners نشان

فصل دوم: مفاهیم پایه و کارهای مرتبط

داد که حتی یک دستور ساده برای فکر کردن گام به گام نیز می تواند بدون ارائه نمونه های آماده، بخش مهمی از این قابلیت را فعال کند. [۵]، [۶]. با این حال، آشکار شدن گام های میانی به خودی خود به معنای استدلال قابل اتکا نیست. از یک سو، گام های میانی ممکن است فقط باز نویسی هایی طولانی از پاسخ نهایی باشند و به پیشبرد واقعی حل کمک نکنند؛ از سوی دیگر، ممکن است زنجیره ای منسجم از نظر زبانی تولید شود که از نظر منطقی دچار خطاست. به همین دلیل، در سال های اخیر تمایز میان تولید صرف زنجیره و تولید زنجیره ای مفید، معتبر و ساختاریافته اهمیت یافته است. این تمایز مسیر را به سوی روش هایی باز کرده که فقط به طولانی تر کردن مسیر فکر اکتفا نمی کنند، بلکه در پی کنترل کیفیت آن، سنجش درستی آن، یا باز طراحی ساختار آن هستند. [۷]، [۱۰]، [۱۱]، [۱۶].

در این چارچوب، می توان گفت استدلال در مدل های زبانی سه بُعد اصلی دارد. بُعد نخست، تولید نمایش میانی^{۱۴} است؛ یعنی مدل بتواند پیش از پاسخ نهایی، گام هایی را عرضه کند که حامل اطلاعات مفید باشند. بُعد دوم، سازمان دهی این نمایش است؛ یعنی این گام ها باید به ترتیب مناسب، با حفظ وابستگی ها و بدون گسست منطقی کنار هم قرار گیرند. بُعد سوم، اعتبار این نمایش است؛ یعنی زنجیره تولید شده باید قابلیت بررسی^{۱۵}، ارزیابی و در صورت لزوم اصلاح داشته باشد. پروژه حاضر از همین برداشت سه مرحله ای از استدلال استفاده می کند و به همین دلیل، تمرکز خود را تنها بر تولید متن بیشتر نگذاشته، بلکه بر ساختار، کنترل و اعتبار مسیر استدلال متمرکز شده است.

۲-۲-۲ استدلال در حل مسئله ریاضی

حل مسئله ریاضی برای مطالعه استدلال بستری مناسب و در عین حال دشوار است. مناسب است زیرا در بسیاری از مسائل ریاضی، به ویژه مسائل کلامی و چند مرحله ای، ساختار حل نسبتاً روشن است: ورودی شامل داده ها، قیود و هدف مسئله است و خروجی باید نتیجه ای باشد که با همه این عناصر سازگار باشد. دشوار است زیرا همین شفافیت، امکان پنهان شدن خطاها را کم می کند. در یک مسئله ریاضی، نادیده گرفتن یک قید، ساختن وابستگی اشتباه، یا استفاده از میان بری نادرست به سرعت به پاسخ غلط منتهی می شود. به همین دلیل، ریاضیات یکی از مناسب ترین میدان ها برای سنجش کیفیت واقعی استدلال

¹⁴ Intermediate Representation

¹⁵ Verification

مدل است. علاوه بر این، در مسائل ریاضی معمولاً تمایز میان دو مرحله به خوبی قابل مشاهده است: نخست فهم ساختار مسئله، و سپس اجرای حل. مدل باید ابتدا بفهمد چه چیزی خواسته شده است، چه مجهول‌هایی واقعاً اهمیت دارند، کدام قیود در ادامه به کار می‌آیند، و چه وابستگی‌هایی میان اجزای مسئله برقرار است. تنها پس از این مرحله است که محاسبه، استنتاج جبری یا شمارش حالت‌ها معنا پیدا می‌کند. بنابراین، اگر مدل در مرحله فهم ساختار شکست بخورد، حتی اجرای محاسبات درست نیز نمی‌تواند آن را نجات دهد. همین ویژگی است که مسائل ریاضی را به بستر مناسبی برای بررسی مرحله برنامه‌ریزی تبدیل می‌کند. [۹]، [۱۰]، [۱۱].

پژوهش Training Verifiers to Solve Math Word Problems [۱۱] از این منظر اهمیت ویژه‌ای دارد، زیرا نشان داد که حتی مدل‌های قدرتمند نیز در استدلال چندمرحله‌ای ریاضی خطاهای جدی دارند و استفاده از داور می‌تواند انتخاب پاسخ بهتر را ممکن سازد. اهمیت این نتیجه برای پروژه حاضر در آن است که نشان می‌دهد ضعف در ریاضیات صرفاً ناشی از ناتوانی محاسباتی نیست، بلکه بخش مهمی از آن به کیفیت مسیر حل مربوط می‌شود. به بیان دیگر، حل مسئله ریاضی فقط مسئله عدد و فرمول نیست، بلکه مسئله ساختن یک مسیر حل معتبر است. در این پروژه، توجه به مسائل ریاضی از همین زاویه شکل گرفته است. تمرکز بر ریاضیات نه به این دلیل بود که دامنه از ابتدا مشخص بوده، بلکه به این دلیل که در مراحل بعدی روشن شد این دامنه برای تحلیل خطاهای ساختاری، تشخیص افت‌های مرحله برنامه‌ریزی، و ارزیابی دقیق کیفیت طرح حل، بستری مناسب‌تر از بسیاری از حوزه‌های دیگر است. در نتیجه، ریاضیات در اینجا نه صرفاً یک موضوع کاربردی، بلکه یک محیط کنترل‌شده برای مطالعه رفتار استدلالی مدل محسوب می‌شود.

۲-۲-۳ ساختار میان‌مرحله‌ای در فرایند استدلال

یکی از مفاهیم کلیدی برای فهم پروژه حاضر، ساختار میان‌مرحله‌ای است. منظور از این ساختار، هر نوع نمایش میانی است که میان ورودی مسئله و پاسخ نهایی قرار می‌گیرد و نقش سازمان‌دهی فرایند استدلال را بر عهده دارد. این نمایش می‌تواند به شکل زنجیره‌ای از گام‌ها، یک طرح کلی، مجموعه‌ای از زیرهدف‌ها، فهرستی از قیود، یا حتی ساختاری شاخه‌ای برای حالت‌های شرطی ظاهر شود. اهمیت این مفهوم در آن است که بسیاری از خطاهای مدل در پاسخ نهایی، در واقع محصول نقص در همین ساختار میانی‌اند. [۵]، [۹]، [۱۰]، [۱۶].

فصل دوم: مفاهیم پایه و کارهای مرتبط

ساختار میان‌مرحله‌ای خوب چند ویژگی دارد. نخست، باید هدف مسئله را به‌روشنی حفظ کند، یعنی معلوم باشد مدل در هر گام به‌دنبال رسیدن به چه خروجی‌ای است. دوم، باید مجهولات واقعی را از داده‌های ثابت جدا کند، زیرا آمیختگی این دو باعث تولید مسیرهای شلوغ و غیرضروری می‌شود. سوم، باید قیود را به‌صورت گزینشی استخراج کند، یعنی فقط آن دسته از اطلاعات مسئله را نگه دارد که در ادامه واقعاً اثرگذارند. چهارم، باید وابستگی‌ها را روشن سازد، برای مثال مشخص کند که کدام کمیت از کدام شرط یا کدام نتیجه میانی به دست می‌آید. [۹]، [۱۰]، [۱۶].

روش‌های مبتنی بر نمونه‌برداری چندمسیره و انتخاب پاسخ نیز در عمل بر همین ایده تکیه دارند. Self-Consistency با نمونه‌گیری چند مسیر و انتخاب پاسخ سازگارتر، عملاً فرض می‌کند که برخی مسیرهای میانی بهتر از بقیه‌اند و انتخاب درست از میان آن‌ها می‌تواند عملکرد را بهبود دهد. Best-of-N نیز همین ایده را در قالب کلی‌تری دنبال می‌کند: چند نامزد تولید می‌شوند و سپس بهترین آن‌ها طبق یک قاعده انتخاب می‌گردد. بنابراین، حتی در روش‌هایی که مستقیماً به برنامه‌ریزی تصریح نمی‌شود، باز هم ساختار میان‌مرحله‌ای و کیفیت آن نقش محوری دارد. [۷]، [۱۱]، [۱۴]، [۱۵].

در پروژه حاضر، این ساختار میان‌مرحله‌ای در نهایت به‌صورت یک طرح حل حداقلی و انسان‌گونه فهم شد؛ طرحی که در آن هدف، مجهولات واقعی، قیود اصلی و اسکلت وابستگی پیش از هرگونه محاسبه تولید می‌شود. این برداشت از ساختار میان‌مرحله‌ای، تفاوت مهمی با زنجیره‌های آزاد و پرگویی دارد، زیرا هدف آن نه تولید متن بیشتر، بلکه تولید سازمان بیشتر است. همین تفاوت، مسیر را از روش‌های صرفاً مولد به سمت روش‌های برنامه‌ریزی‌محور سوق می‌دهد.

۲-۲-۴ برنامه‌ریزی، تجزیه مسئله و حل مرحله‌ای

برنامه‌ریزی^{۱۶} به معنای ساختن یک نقشه حداقلی برای پیمایش مسئله است. این نقشه باید نشان دهد که برای رسیدن به پاسخ، چه مؤلفه‌هایی واقعاً لازم‌اند، کدام روابط نقش مرکزی دارند، و ترتیب مناسب پیشروی چیست. چنین برداشتی از برنامه‌ریزی با رویکردهای کلاسیک حل مسئله نیز سازگار است: در مسائل دشوار، انسان‌ها معمولاً پیش از ورود به محاسبه، صورت مسئله را بازسازمان‌دهی می‌کنند، متغیرها و قیود را جدا می‌کنند، و ابتدا مسیر را می‌فهمند، سپس آن را اجرا می‌کنند.

¹⁶ Planning

فصل دوم: مفاهیم پایه و کارهای مرتبط

در ادبیات مدل‌های زبانی، **Plan-and-Solve Prompting** یکی از روشن‌ترین نمونه‌های تبدیل این شهود به یک روش صریح است. این پژوهش نشان می‌دهد که اگر مدل ابتدا مسئله را به زیرگام‌های معنادار تقسیم کند و سپس حل را مطابق آن برنامه پیش ببرد، بخشی از خطاهای ناشی از جالفتادن گام‌ها و سوءبرداشت معنایی کاهش می‌یابد. اهمیت این ایده برای پروژه حاضر در آن است که برنامه‌ریزی را از یک قابلیت فرعی به یک مرحله مستقل در فرایند حل تبدیل می‌کند. [۹].

تجزیه مسئله نیز دقیقاً در همین چارچوب معنا پیدا می‌کند. تجزیه خوب لزوماً به معنای خرد کردن مسئله به تعداد زیادی زیرمسئله نیست. در بسیاری از موارد، تجزیه بیش از حد می‌تواند خود به منبع خطا تبدیل شود. تجزیه مؤثر آن است که فقط اجزای لازم را استخراج کند، مرز هر جزء را روشن نگه دارد، و وابستگی میان اجزا را به‌درستی حفظ کند. این تعریف با رویکردی که در پروژه حاضر برای مرحله برنامه‌ریزی در نظر گرفته شد سازگار است: هدف، تولید کمینه‌ای از ساختار لازم برای حل است، نه تولید طولانی‌ترین توصیف ممکن از فرایند فکر. حل مرحله‌ای نیز پیامد طبیعی همین نگاه است. وقتی برنامه‌ریزی و تجزیه مسئله انجام شد، اجرای حل می‌تواند به‌صورت گام‌به‌گام پیش برود و هر گام در صورت نیاز بررسی شود. همین منطق باعث شده است در ادبیات جدید، روش‌هایی که به اجرای مرحله‌ای همراه با داوری، بازبینی یا پالایش تکیه دارند، توجه زیادی جلب کنند. [۱۰]، [۱۶].

۳-۲ روش‌های بهبود استدلال در استنتاج

روش‌های بهبود استدلال در زمان استنتاج را می‌توان از منظر محل مداخله به چند خانواده اصلی تقسیم کرد. خانواده نخست، روش‌هایی هستند که بدون تغییر پارامترهای مدل، فقط نحوه درخواست از مدل را عوض می‌کنند؛ مانند **Chain-of-Thought Prompting**، **Zero-shot Prompting** و **Plan-and-Solve Prompting**. در این روش‌ها فرض بر آن است که بخشی از توان استدلالی مدل در خود آن وجود دارد، اما تنها در صورتی آشکار می‌شود که ورودی به‌شکل مناسبی صورت‌بندی شود. [۵]، [۶]، [۹].

خانواده دوم، روش‌هایی‌اند که محاسبه بیشتری را در زمان استنتاج صرف می‌کنند. در این‌جا به‌جای یک خروجی واحد، چند مسیر یا چند پاسخ تولید می‌شود و سپس با استفاده از سازگاری، رأی‌گیری یا داوری، یک خروجی نهایی انتخاب

می‌گردد. سازگاری با خود^{۱۷} نمونه شناخته‌شده این رویکرد است و Best-of-N صورت کلی‌تر آن را نشان می‌دهد. ایده اصلی این خانواده آن است که یک پاسخ واحد ممکن است به شدت به تصادف نمونه‌گیری وابسته باشد، اما اگر چند مسیر مستقل تولید شوند، می‌توان از توافق یا کیفیت نسبی آن‌ها برای انتخاب بهتر استفاده کرد. [۷]، [۱۱]، [۱۴]، [۱۵]. در همین چارچوب، مقیاس‌دهی در زمان استنتاج به این معناست که به جای بزرگ‌تر کردن خود مدل، بودجه محاسباتی در زمان پاسخ‌گویی افزایش یابد. این افزایش می‌تواند به صورت طولانی‌تر کردن مسیر فکر، افزایش تعداد نمونه‌ها، افزودن داوری، یا اجرای پالایش‌های بیشتر رخ دهد. پژوهش ۵۱ نشان می‌دهد که حتی روش‌های ساده برای کنترل بودجه استنتاج، مانند وادار کردن مدل به ادامه فکر یا قطع زودهنگام آن، می‌توانند اثر محسوسی بر عملکرد استدلالی داشته باشند و این ایده را به صورت باز و قابل تکرار مطالعه می‌کند. [۱۳].

خانواده سوم، روش‌های مبتنی بر داوری و اعتبارسنجی‌اند^{۱۸}. در این روش‌ها یک پاسخ یا زنجیره تولید می‌شود، اما یک جزء دیگر وظیفه دارد کیفیت آن را ارزیابی کند. این جزء ممکن است یک داور آموزش‌دیده باشد، یا خود مدل با یک پرامپت دیگر نقش داور را ایفا کند. پژوهش Let's Verify Step by Step [۱۰] این ایده را در قالب مدل پاداش‌فرایندی^{۱۹} دنبال می‌کند و نشان می‌دهد که برچسب‌گذاری درستی گام‌های میانی می‌تواند در انتخاب پاسخ نهایی سودمند باشد. پژوهش Zero-Shot Verification-guided Chain of Thoughts [۱۶] نیز همین ایده را در حالت بدون آموزش تخصصی داور بررسی می‌کند و نشان می‌دهد که می‌توان از داوری مرحله‌ای در محیط کاملاً صفرنمونه استفاده کرد.

خانواده چهارم، روش‌هایی هستند که تلاش می‌کنند شکاف میان راهبرد استنتاج و فرایند آموزش را کمتر کنند. Inference-Aware Fine-Tuning نمونه روشن این جهت‌گیری است: اگر قرار است در زمان استنتاج از Best-of-N یا راهبردهای چندنمونه‌ای استفاده شود، خود مدل نیز باید با در نظر گرفتن همین راهبرد تنظیم دقیق شود. این ایده از نظر مفهومی مهم است، زیرا نشان می‌دهد استراتژی استنتاج و استراتژی تنظیم را نمی‌توان کاملاً مستقل از یکدیگر دید. [۱۴].

¹⁷ Self-Consistency

¹⁸ Voting and Verification

¹⁹ Process Reward Model (PRM)

فصل دوم: مفاهیم پایه و کارهای مرتبط

خانواده پنجم، روش‌های پس‌آموزش بسیار سبک و کم‌هزینه‌اند که می‌کوشند با کمترین داده و کمترین به‌روزرسانی، رفتار مدل را در جهت مطلوب جابه‌جا کنند. One-shot Entropy Minimization [۱۷] نمونه جالب این جهت‌گیری است که نشان می‌دهد حتی با داده بسیار اندک و بهینه‌سازی کوتاه نیز می‌توان رفتار مدل را تغییر داد. این دسته از پژوهش‌ها برای پروژه حاضر اهمیت دارند، زیرا از نظر عملی با محدودیت‌های محاسباتی و زمانی پروژه سازگارترند و نشان می‌دهند که برای بهبود رفتار مدل همیشه به آموزش‌های سنگین نیاز نیست.

در کنار این خانواده‌ها، یک دسته از پژوهش‌ها به‌جای تکیه اصلی بر پرامپت یا داوری، خود ساختار محاسباتی را بازطراحی می‌کنند. Hierarchical Reasoning Model نمونه‌ای از این دسته است که دو ماژول با مقیاس زمانی متفاوت را برای برنامه‌ریزی انتزاعی و محاسبه جزئی ترکیب می‌کند. هرچند این کار دقیقاً در چارچوب متعارف مدل‌های زبانی محاوره‌ای قرار نمی‌گیرد، اما از منظر مفهومی برای پروژه حاضر مهم است، زیرا نشان می‌دهد تفکیک میان برنامه‌ریزی آهسته و محاسبه سریع، صرفاً یک ترفند پرامپتی نیست و می‌تواند به‌صورت معماری نیز مدل شود. [۱۸].

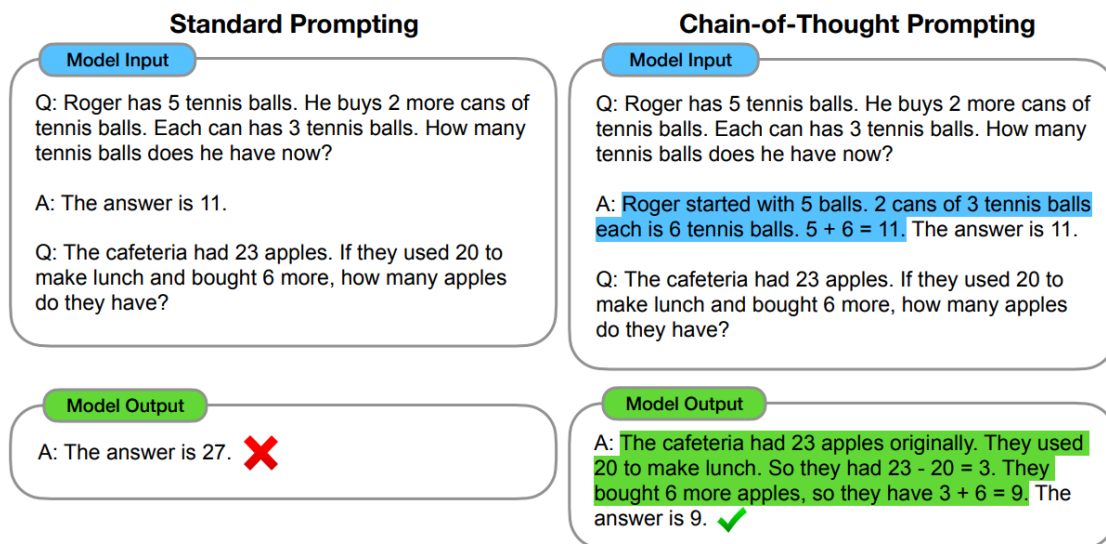
در نتیجه، روش‌های بهبود استدلال در زمان استنتاج را نمی‌توان به یک تکنیک واحد محدود کرد. برخی روش‌ها به فعال‌سازی بهتر ظرفیت موجود مدل متکی‌اند، برخی به صرف محاسبه بیشتر، برخی به افزوده شدن یک داور، و برخی به هم‌راستاسازی آموزش و استنتاج. آنچه برای این پروژه مهم بود، نه صرفاً بیشترین دقت ممکن در هر شرایط، بلکه یافتن ترکیبی از این ایده‌ها بود که از نظر محاسباتی شدنی، از نظر تحلیلی قابل کنترل، و از نظر روش‌شناختی مناسب برای مدل‌های کوچک باشد. [۹]، [۱۰]، [۱۴]، [۱۵]، [۱۷]، [۱۸].

۴-۲ کارهای مرتبط

۱-۴-۲ روش‌های مبتنی بر پرامپت

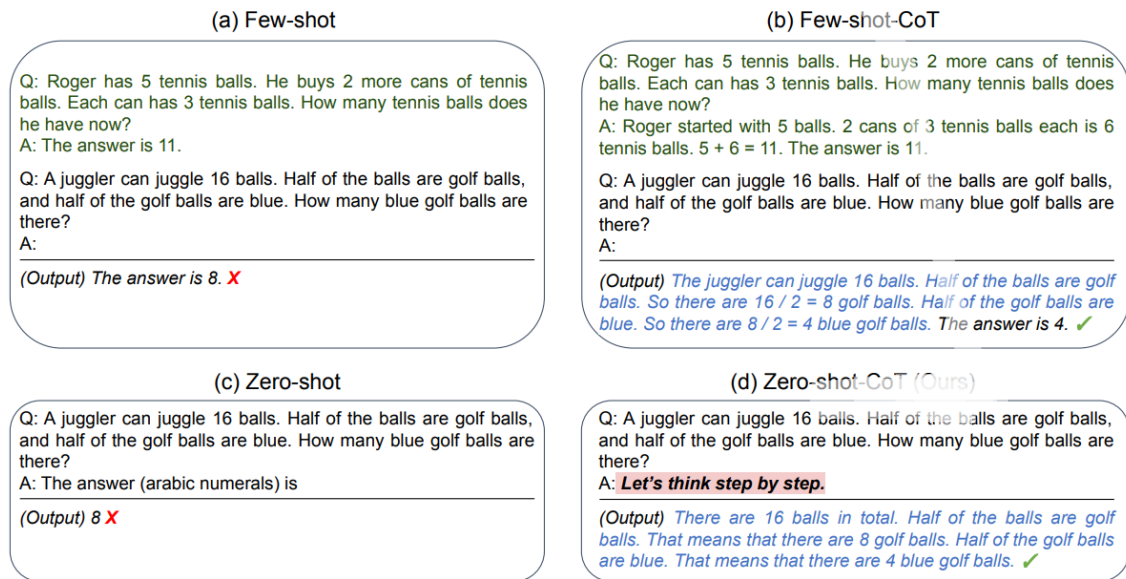
نخستین خانواده از کارهای مرتبط، پژوهش‌هایی هستند که بدون تغییر پارامترهای مدل، تنها با بازطراحی ورودی، کیفیت استدلال را بهبود می‌دهند. Chain-of-Thought Prompting نقطه عطف این خط پژوهش است، زیرا نشان داد ارائه نمونه‌هایی از گام‌های میانی به مدل می‌تواند عملکرد آن را در طیفی از وظایف استدلالی بالا ببرد. اهمیت این کار فقط در

نتایج تجربی آن نیست، بلکه در این نکته است که استدلال را از دل خود فرایند تولید متن بیرون کشید و به موضوعی قابل مطالعه تبدیل کرد. [۵].



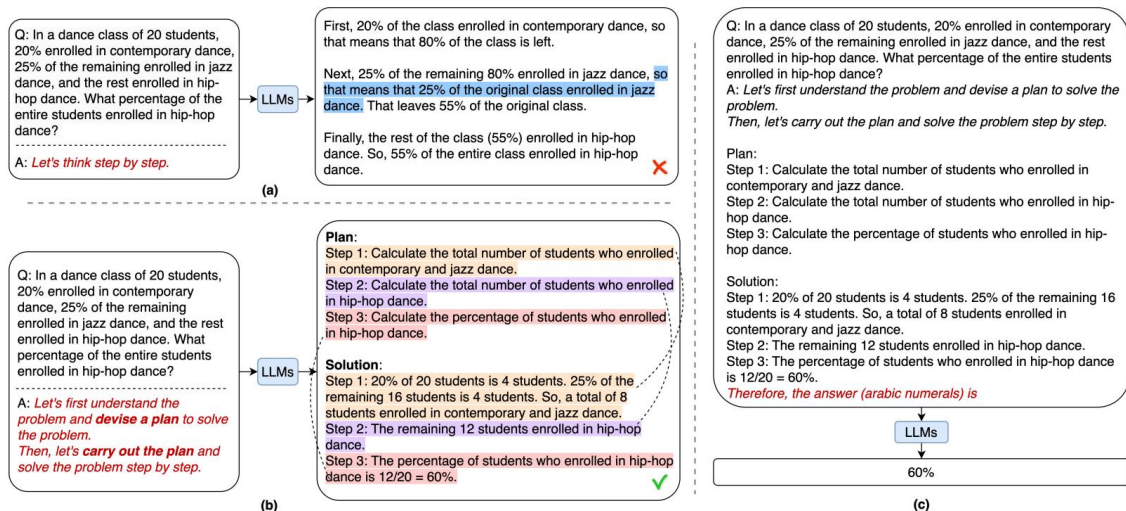
شکل 2-۱ پرامپت مبتنی بر CoT

Zero-shot Prompting این خط را ساده تر و عمومی تر کرد. در این رویکرد، به جای ارائه مثال های آماده، تنها یک دستور کلی برای فکر کردن مرحله به مرحله به مدل داده می شود. نتیجه این کار آن بود که بخش مهمی از قابلیت استدلالی مدل بدون نیاز به طراحی نمونه های دستی فعال شد. این نکته برای پروژه حاضر مهم است، زیرا نشان می دهد حتی پیش از ورود به مراحل پیچیده تر، شکل پرامپت می تواند در کنترل رفتار استدلالی مدل اثرگذار باشد. [6].



شکل 2-2 پرامپت‌های Zero-shot-CoT و Zero-shot.Few-shot.CoT

Plan-and-Solve Prompting گام بعدی این مسیر است. در این پژوهش، به جای اینکه مدل صرفاً به فکر کردن گام‌به‌گام دعوت شود، از آن خواسته می‌شود ابتدا یک برنامه برای حل بسازد و سپس حل را مطابق همان برنامه اجرا کند. افزوده شدن این مرحله مستقل برنامه‌ریزی برای پروژه حاضر بسیار تعیین‌کننده بود، زیرا به جای نگاه به زنجیره فکر به عنوان یک متن پیوسته، آن را به دو بخش کارکردی قابل تفکیک تقسیم می‌کرد: بخش سازمان‌دهی و بخش اجرا. [9].



شکل 2-3 پرامپت مبتنی بر Plan-and-Solve

این خانواده از کارها از نظر سادگی و هزینه اجرایی مزیت دارند، اما محدودیت آن‌ها نیز روشن است. اگر مدل در ذات خود برای یک نوع مسئله ضعف جدی داشته باشد، بازطراحی پرامپت همیشه کافی نیست. به بیان دیگر، پرامپت می‌تواند رفتار مدل را بهتر آشکار کند یا تا حدی سامان دهد، اما تضمین نمی‌کند که ساختار تولیدشده واقعاً معتبر، کامل یا پایدار باشد. همین محدودیت یکی از دلایلی بود که در پروژه حاضر، روش‌های مبتنی بر پرامپت نقطه شروع در نظر گرفته شدند، نه نقطه پایان.

۲-۴-۲ روش‌های مبتنی بر نظرسنجی

در این بخش، عنوان نظرسنجی را می‌توان به صورت فنی‌تر به خانواده روش‌های مبتنی بر رأی‌گیری، سازگاری و داوری تعبیر کرد. این روش‌ها بر این فرض استوارند که یک پاسخ منفرد لزوماً بهترین نماینده رفتار مدل نیست و می‌توان با تولید چند پاسخ یا چند مسیر و سپس انتخاب از میان آن‌ها، به عملکرد بهتری رسید. سازگاری با خود نمونه کلاسیک این رویکرد است: به جای تکیه بر یک خروجی حریصانه، چند مسیر مستقل تولید می‌شوند و پاسخی که بیشترین سازگاری را با سایر خروجی‌ها دارد انتخاب می‌شود. [۷]

پژوهش‌های مبتنی بر داور یک گام فراتر می‌روند. در اینجا صرف توافق اکثریت ملاک نیست، بلکه یک داور یا معیار بیرونی کیفیت هر نامزد را می‌سنجد. `Let's Verify` و `Training Verifiers to Solve Math Word Problems` [۱۱] و `Step by Step` [۱۰] دو نمونه مهم از این رویکردها هستند. اولی نشان می‌دهد که می‌توان داوری آموخته‌شده را برای انتخاب پاسخ‌های بهتر در مسائل ریاضی به کار برد، و دومی نشان می‌دهد که حتی ارزیابی سطح گام‌های میانی نیز می‌تواند در انتخاب پاسخ نهایی مفید باشد.

در پژوهش‌های جدیدتر، این ایده با کارایی محاسباتی نیز ترکیب شده است. `Inference-Aware Fine-Tuning` [۱۴] از منظر آموزشی به راهبرد چندنمونه‌ای نگاه می‌کند و `ModelSwitch` در مقاله `Do We Truly Need So Many Samples?` [۱۵] نشان می‌دهد که می‌توان از چند مدل با هزینه کمتر استفاده کرد و بر اساس سازگاری، میان آن‌ها جابه‌جا شد. اهمیت این دسته برای پروژه حاضر در آن است که نشان می‌دهد کیفیت پاسخ فقط تابع کیفیت یک مسیر واحد نیست، بلکه تابع نحوه استفاده از چند مسیر و قاعده انتخاب میان آن‌ها نیز هست. در عین حال، این روش‌ها هزینه خود را دارند. هرچه تعداد نمونه‌ها یا مراحل داوری بیشتر شود، زمان استنتاج و مصرف منابع بیشتر می‌شود. برای پروژه حاضر که

فصل دوم: مفاهیم پایه و کارهای مرتبط

از ابتدا با محدودیت منابع روبه‌رو بود، این نکته مهم بود؛ به همین دلیل این خانواده از روش‌ها بیشتر از منظر مفهومی و برای درک نقش سازگاری و داوری مطالعه شدند، نه به‌عنوان راه‌حل نهایی و بدون قید.

۲-۴-۳ روش‌های مبتنطی بر تنظیم و مقیاس‌دهی

دسته سوم از کارهای مرتبط، پژوهش‌هایی‌اند که یا از راه تنظیم مدل، یا از راه کنترل بودجه استنتاج، در پی بهبود استدلال‌اند. Inference-Aware Fine-Tuning [۱۴] نمونه مهمی است، زیرا استدلال می‌کند که اگر در زمان استنتاج از Best-of-N یا راهبردهای مشابه استفاده می‌کنیم، تنظیم مدل نیز باید با در نظر گرفتن همین راهبرد انجام شود. این کار از نظر مفهومی برای پروژه حاضر مهم بود، چون نشان می‌داد مرز میان آموزش و استنتاج کاملاً صلب نیست و می‌توان بخشی از رفتار مطلوب استنتاجی را به مرحله تنظیم منتقل کرد.

پژوهش ۵۱ [۱۳] از سوی دیگر نشان می‌دهد که مقیاس‌دهی در زمان استنتاج می‌تواند حتی با مکانیزم‌های نسبتاً ساده هم اثرگذار باشد، به شرط آنکه بودجه استنتاج به‌شکل کنترل‌شده مصرف شود. این نوع نگاه برای پروژه حاضر مهم بود، زیرا با محدودیت‌های سخت‌افزاری سازگارتر از بسیاری از راهبردهای سنگین بود و نشان می‌داد بهبود استدلال لزوماً مستلزم بازآموزی کامل یا استفاده از مدل‌های بسیار بزرگ نیست.

One-shot Entropy Minimization [۱۷] از زاویه‌ای دیگر همین نکته را تقویت می‌کند. ایده اصلی این کار آن است که حتی با داده بسیار محدود و یک فرایند بهینه‌سازی کوتاه می‌توان رفتار مدل را در جهت مطلوب سوق داد. اگرچه این پژوهش مستقیماً بر مسئله برنامه‌ریزی متمرکز نیست، اما از نظر پروژه حاضر مهم است، زیرا نشان می‌دهد در فضای پس‌آموزش نیز راهبردهای کم‌هزینه و قابل‌اجرا وجود دارند.

در همین بخش، می‌توان به پژوهش Hierarchical Reasoning Model [۱۸] نیز اشاره کرد. این کار از جنس تنظیم سبک‌وزن روی مدل زبانی رایج نیست، اما از نظر معماری نشان می‌دهد که تفکیک برنامه‌ریزی انتزاعی و محاسبه جزئی می‌تواند در سطح مدل نیز موضوعیت داشته باشد. در واقع، اگر بخش زیادی از خطای استدلال ناشی از آمیختگی تصمیم‌گیری کلان و محاسبه خرد باشد، آنگاه تفکیک این دو نه فقط در پرامپت، بلکه در معماری نیز می‌تواند سودمند باشد. برای پروژه حاضر، این کار بیشتر جنبه الهام‌بخش داشت تا مبنای مستقیم پیاده‌سازی.

سرانجام، روش‌های تنظیم دقیق سبک‌وزن مانند LORA نیز در این خانواده اهمیت دارند. هرچند LORA خود یک روش ویژه برای استدلال نیست، اما از آن‌جا که امکان تنظیم کارآمد مدل با تعداد پارامترهای آموزش‌پذیر بسیار کمتر را فراهم می‌کند، برای پروژه‌هایی که قصد دارند فقط بخشی از رفتار مدل را تغییر دهند بسیار مناسب است. از این رو، این دسته از روش‌ها در پروژه حاضر نه به‌عنوان راهکار مفهومی اصلی استدلال، بلکه به‌عنوان ابزار اجرایی بالقوه برای انتقال رفتار مطلوب به مدل کوچک اهمیت پیدا کردند. [۱۲].

۴-۴-۲ جمع‌بندی کارهای مبتنی مرتبط با پروژه

مرور کارهای مرتبط نشان می‌دهد که پژوهش‌های موجود را می‌توان بر حسب نوع مداخله در سه مسیر کلی خلاصه کرد: فعال‌سازی بهتر استدلال موجود با پرامپت، بهبود انتخاب پاسخ از طریق چندنمونه‌گیری و داوری، و انتقال یا پایداری رفتار مطلوب از طریق تنظیم و مقیاس‌دهی. هیچ‌یک از این مسیرها به‌تنهایی پاسخ کامل مسئله نیستند، اما هر یک بخش مهمی از پازل را روشن می‌کنند.

از منظر پروژه حاضر، چند نکته از این مرور برجسته می‌شود. نخست، پژوهش‌های بر پایه مهندسی پرام نشان می‌دهند که ساختاردهی ورودی اهمیت دارد، اما این ساختاردهی باید فراتر از دستورهای کلی باشد. دوم، پژوهش‌های داوری و چندنمونه‌گیری نشان می‌دهند که کیفیت پاسخ را می‌توان از طریق انتخاب بهتر بالا برد، اما این کار هزینه محاسباتی دارد. سوم، پژوهش‌های تنظیم و مقیاس‌دهی نشان می‌دهند که در صورت تعریف درست هدف، می‌توان بخشی از رفتار مطلوب را با هزینه‌ای قابل قبول به مدل منتقل کرد. چهارم، کارهای معماری جدیدتر نیز بر اهمیت جداسازی نقش برنامه‌ریزی و محاسبه صحنه می‌گذارند. به همین دلیل، برای پروژه حاضر رویکردی مناسب‌تر به نظر رسید که این بینش‌ها را به‌صورت ترکیبی به کار گیرد: ابتدا ساختار استدلالی مسئله در قالب یک برنامه حداقلی استخراج شود، سپس حل بر مبنای آن انجام گیرد، و در صورت نیاز همان مرحله برنامه‌ریزی به‌صورت مستقل تحلیل و بهبود داده شود. این انتخاب نه محصول یک تصمیم ناگهانی، بلکه نتیجه طبیعی مرور ادبیات و مقایسه مسیرهای ممکن بود.

۵-۲ جمع‌بندی

در این فصل، مفاهیم پایه لازم برای فهم مسئله و مسیر پژوهش تشریح شد و نشان داده شد که استدلال در مدل‌های زبانی صرفاً معادل پاسخ صحیح نیست، بلکه به کیفیت نمایش‌های میانی، سازمان‌دهی گام‌ها و قابلیت بررسی آن‌ها وابسته است. همچنین توضیح داده شد که چرا حل مسئله ریاضی بستر مناسبی برای مطالعه این مسئله است و چگونه ساختارهای میان‌مرحله‌ای، به‌ویژه برنامه‌ریزی و تجزیه مسئله، می‌توانند به‌عنوان نقطه مداخله‌ای معنادار برای بهبود عملکرد در نظر گرفته شوند. در ادامه، خانواده‌های اصلی روش‌های بهبود استدلال در زمان استنتاج و مهم‌ترین کارهای مرتبط مرور شد. جمع‌بندی این بررسی‌ها نشان داد که برای پروژه‌ای با محدودیت منابع، رویکردی که بر ساختاردهی فرایند استدلال، کنترل مرحله برنامه‌ریزی و امکان بهبود سبک‌وزن همان مرحله تکیه کند، نسبت به بسیاری از مسیرهای دیگر مناسب‌تر است. بر همین مبنا، فصل بعدی به تشریح روش پیشنهادی، محدودشدن دامنه مسئله، و نحوه طراحی و پیاده‌سازی مسیر انتخاب‌شده خواهد پرداخت.

نام خانواده روش	مقاله	تغییر در	هزینه در زمان استنتاج	نیاز به تنظیم دقیق	مزیت اصلی	محدودیت اصلی
روش‌های مبتنی بر پرامپت	[5]، [6]	صورت‌بندی ورودی و نحوه درخواست از مدل	پایین	ندارد	فعال‌سازی ظرفیت نهفته استدلالی بدون تغییر مدل	وابسته به کیفیت پرامپت؛ تضمین ساختار معتبر ندارد
برنامه‌ریزی پیش از حل	[9]	تفکیک مرحله برنامه‌ریزی از اجرا	پایین تا متوسط	ندارد	کاهش خطاهای ناشی از جابجاندن گام‌ها؛ ایجاد ساختار میانی	همچنان وابسته به کیفیت تولید مدل

فصل دوم: مفاهیم پایه و کارهای مرتبط

نمونه‌گیری چندمسیره و سازگاری	[7]	مرحله انتخاب پاسخ از میان چند مسیر	متوسط تا بالا وابسته به (تعداد نمونه	ندارد	کاهش حساسیت به تصادف نمونه‌گیری	افزایش هزینه استنتاج
و بازرتبه‌بندی Best-of-N	[14]، [15]	تولید چند نامزد و انتخاب بهترین	بالا	گاهی دارد	بهبود دقت از طریق انتخاب هوشمندانه	هزینه محاسباتی بالا
داوری و اعتبارسنجی	[11]، [10]، [16]	افزودن مؤلفه ارزیاب به خروجی یا گام‌های میانی	متوسط تا بالا	در برخی موارد دارد	افزایش قابلیت اعتماد پاسخ	پیچیدگی پیاده‌سازی و هزینه بیشتر
مقیاس‌دهی در زمان استنتاج	[13]	افزایش بودجه محاسباتی در پاسخ‌گویی	متغیر (قابل تنظیم)	ندارد	بهبود عملکرد بدون تغییر پارامترها	افزایش زمان پاسخ‌گویی
تنظیم آگاه از راهبرد استنتاج	[14]	مرحله آموزش با در نظر گرفتن راهبرد انتخاب	بدون افزایش مستقیم در استنتاج، اما نیازمند تنظیم	دارد	هم‌راستاسازی آموزش و استنتاج	نیاز به داده و تنظیم مجدد
پس‌آموزش سبک‌وزن	[12]، [17]	تنظیم بخشی از پارامترها یا بهینه‌سازی کوتاه	بدون افزایش مستقیم	دارد (کم‌هزینه)	هزینه پایین و عملی برای مدل کوچک	بهبود محدود در صورت ضعف ساختاری عمیق
تفکیک معماری برنامه‌ریزی و محاسبه	[18]	سطح معماری مدل	وابسته به طراحی	دارد	جداسازی تصمیم‌گیری کلان و محاسبه جزئی	پیچیدگی طراحی و پیاده‌سازی

فصل سوم: روش پیشنهادی و نتیجه‌گیری

۱-۳ مقدمه

در فصل‌های پیشین، ابتدا مسئله استدلال در مدل‌های زبانی از منظر مفهومی و ادبیات پژوهش بررسی شد و سپس نشان داده شد که برای مدل‌های کوچک‌تر، تقویت استدلال بیش از آن که متکی به افزایش اندازه مدل باشد، به نحوه سازمان‌دهی فرایند استنتاج وابسته است. مرور کارهای مرتبط نیز نشان داد که رویکردهای مبتنی بر برنامه‌ریزی، تجزیه مسئله، برنامه گام‌به‌گام و تنظیم سبک‌وزن، برای مدل‌های زبانی کوچک از پتانسیل بالایی برخوردارند. بر همین مبنا، اساس روش انتخاب‌شده در این پروژه بر جداسازی مرحله برنامه‌ریزی از مرحله حل بنا شد؛ به این معنا که مدل ابتدا باید ساختار حل را بسازد و پس از آن وارد اجرای حل شود. در این فصل، این ایده از حالت مفهومی به سطح اجرایی می‌رسد. ابتدا مدل و دامنه مسئله انتخاب‌شده معرفی می‌شوند، سپس معماری پیشنهادی در سطح کلان تشریح می‌شود، و در ادامه زمینه ورود به تحلیل خطا، ساخت داده و بهبود مدل فراهم می‌گردد.

۲-۳ مدل، دامنه مسائل و معماری پیشنهادی

در این بخش، مؤلفه‌های اصلی طراحی عملی پروژه معرفی می‌شوند. نخست توضیح داده می‌شود که چرا دامنه حل مسئله ریاضی و مدل Qwen۳-۱.۷B برای این پروژه انتخاب شدند. سپس معماری پیشنهادی در سطح کلان تشریح می‌شود؛ معماری‌ای که از تحلیل خطای مدل آغاز می‌شود، به خوشه‌بندی الگوهای تکرارشونده خطا می‌رسد، برای هر خوشه داده‌ای هدفمند می‌سازد، و در نهایت از تنظیم دقیق سبک‌وزن برای بهبود مرحله برنامه‌ریزی استفاده می‌کند.

۱-۲-۳ انتخاب مدل و دامنه مسئله

برای آن که پروژه از مرحله مطالعه مفهومی به یک مسیر قابل اجرا برسد، لازم بود یک دامنه مشخص برای ارزیابی و بهبود انتخاب شود. در این پروژه، دامنه حل مسئله ریاضی انتخاب شد. یکی از دلایل این انتخاب، آشنایی قبلی انجام‌دهنده پروژه با ساختار مسائل ریاضی و امکان تحلیل دستی کیفیت خروجی‌ها بود. این نکته از نظر عملی مهم است، زیرا در پروژه‌ای که بخش مهمی از آن بر تحلیل خطا، ارزیابی کیفی و بازبینی داده استوار است، انتخاب دامنه‌ای که پژوهشگر بر منطق درونی آن تسلط داشته باشد، فرایند بررسی را دقیق‌تر و قابل‌اتکاتر می‌کند. افزون بر این، مسائل ریاضی به‌طور طبیعی بستری مناسب

برای ارزیابی استدلال‌اند، زیرا در آن‌ها هدف، قیود، مجهولات و مسیر رسیدن به پاسخ نسبتاً شفاف‌تر از بسیاری از دامنه‌های دیگر است و خطاهای ساختاری در آن‌ها راحت‌تر آشکار می‌شوند [۹]. از سوی دیگر، در مرور ادبیات انجام‌شده برای این پروژه، کار مستقیمی که به‌طور خاص بر افزودن قابلیت برنامه‌ریزی گام به گام به یک مدل زبانی کوچک در دامنه ریاضی و بهبود آن از طریق تنظیم دقیق متمرکز باشد، مشاهده نشد. بنابراین این دامنه، هم از نظر تحلیلی مناسب بود و هم از نظر پژوهشی ظرفیت بررسی بیشتری داشت.

پس از انتخاب دامنه، لازم بود مدلی انتخاب شود که از یک‌سو از نظر کیفیت پایه در استدلال و ریاضیات ضعیف و نامناسب نباشد، و از سوی دیگر با محدودیت‌های سخت‌افزاری پروژه سازگار بماند. برای این منظور، مدل Qwen۳-۱.۷B انتخاب شد. بر اساس گزارش فنی Qwen۳ و کارت مدل رسمی آن، این مدل یک مدل زبانی علی با ۱/۷ میلیارد پارامتر است که حدود ۳۰۰ میلیون پارامتر آن مربوط به لایه‌های امبدینگ^{۲۰} می‌باشد. این مدل ۲۸ لایه دارد، و طول پنجره زمینه^{۲۱} آن ۳۲۷۶۸ توکن است. همچنین خانواده Qwen۳ به‌طور کلی با تأکید بر استدلال، تبعیت از دستور، توانمندی عاملی و پشتیبانی چندزبانه توسعه یافته است و گزارش فنی آن تصریح می‌کند که این خانواده در بنچمارک‌های ریاضی و کدنویسی نسبت به نسل‌های پیشین خود بهبود قابل‌توجهی داشته است. [۱۹]

یکی از ویژگی‌های مهم Qwen۳-۱.۷B، پشتیبانی هم‌زمان از حالت تفکر^{۲۲} و حالت بدون تفکر^{۲۳} در یک مدل واحد است. طبق گزارشات رسمی، این مدل به‌صورت پیش‌فرض می‌تواند در حالت تفکر اجرا شود و در این وضعیت، خروجی میانی خود را در بلوک‌های ویژه تولید کند. در عین حال، یک کلید صریح برای غیرفعال کردن این رفتار نیز در اختیار کاربر قرار می‌گیرد و در این حالت، مدل بدون تولید بلوک تفکر پاسخ می‌دهد. در این پروژه، این قابلیت عمداً غیرفعال شد. دلیل این تصمیم آن بود که هدف پروژه، استفاده از استدلال آماده و درونی مدل نبود، بلکه تحمیل و آموزش یک الگوی برنامه‌ریزی به مدل در حالت عادی و بدون اتکا به سازوکار تفکر از پیش تعبیه‌شده بود. به بیان دیگر، مسئله این نبود که یک مدل دارای قابلیت تفکر را صرفاً فعال کنیم، بلکه مسئله این بود که ببینیم آیا می‌توان یک مدل کوچک را در حالت بدون تفکر به‌گونه‌ای

²⁰ Embedding

²¹ Context window

²² Thinking Mode

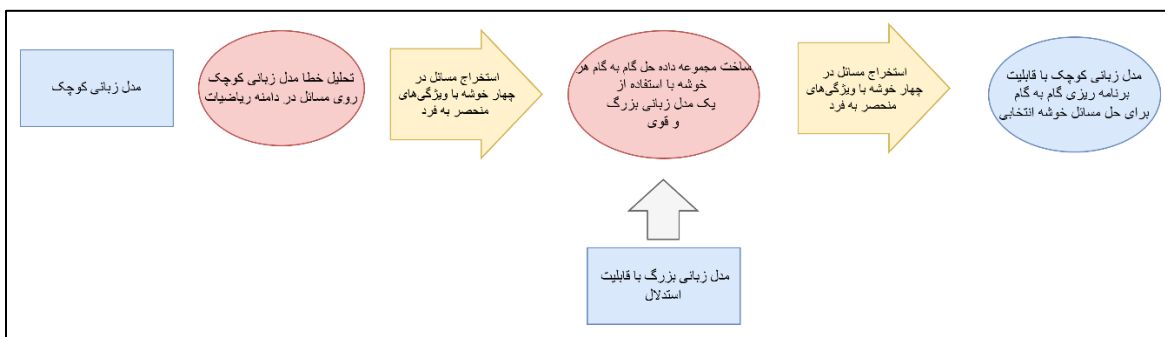
²³ Non-Thinking Mode

هدایت و سپس بهبود داد که پیش از حل، یک طرح حل مناسب تولید کند. از این منظر، غیرفعال کردن حالت تفکر شرط لازم برای خالص نگه‌داشتن مداخله پژوهش بود

در انتخاب این مدل، ملاحظات اجرایی نیز در آن نقش جدی داشتند. پروژه بر روی یک محیط سخت‌افزاری شخصی با کارت گرافیکی RTX ۳۰۶۰ و ۶ گیگابایت حافظه پیش می‌رفت. بنابراین انتخاب یک مدل بزرگ‌تر، نه‌تنها اجرای آزمایش‌ها را ناممکن می‌کرد، بلکه مرحله بعدی پروژه، یعنی تنظیم دقیق سبک‌وزن، را نیز با مانع جدی روبه‌رو می‌ساخت-Qwen۳. از این جهت نقطه تعادل مناسبی فراهم می‌کرد: به‌اندازه‌ای کوچک بود که بتوان در محیط پروژه آن را بارگذاری، آزمون و برای مراحل بعدی روی آن برنامه‌ریزی کرد، و در عین حال به‌اندازه‌ای توانمند بود که ورود به دامنه ریاضی و تحلیل خطا روی خروجی‌های آن معنا داشته باشد. در نتیجه، انتخاب این مدل حاصل جمع سه ملاحظه بود: سازگاری با سخت‌افزار پروژه، برخورداری از ظرفیت پایه مناسب برای مسائل ریاضی، و امکان حرکت به سمت تنظیم دقیق در مراحل بعدی.

۳-۲-۲ معماری پیشنهادی

معماری پیشنهادی این پروژه بر این فرض بنا شده است که در بسیاری از مسائل چندمرحله‌ای، خطای مدل پیش از آن‌که ناشی از محاسبه نهایی باشد، ناشی از ضعف در ساختن مسیر حل است. بر همین اساس، معماری پروژه به‌جای آن‌که مستقیماً بر کل فرایند پاسخ‌گویی مداخله کند، مرحله برنامه‌ریزی را به‌عنوان نقطه اصلی مداخله در نظر می‌گیرد. در این برداشت، برنامه‌ریزی یک زنجیره فکر بلند و آزاد نیست، بلکه یک بازنمایی حداقلی و ساختاریافته از مسیر حل است؛ بازنمایی‌ای که باید هدف مسئله، مجهولات واقعی، قیود مؤثر، و رابطه‌های وابستگی یا شاخه‌های شرطی را پیش از هرگونه حل عددی مشخص کند. به این ترتیب، معماری پیشنهادی از همان ابتدا حول یک تفکیک کارکردی شکل می‌گیرد: ساختن برنامه، و سپس حل بر پایه آن برنامه.



شکل ۳-۱ خط لوله روش پیشنهادی

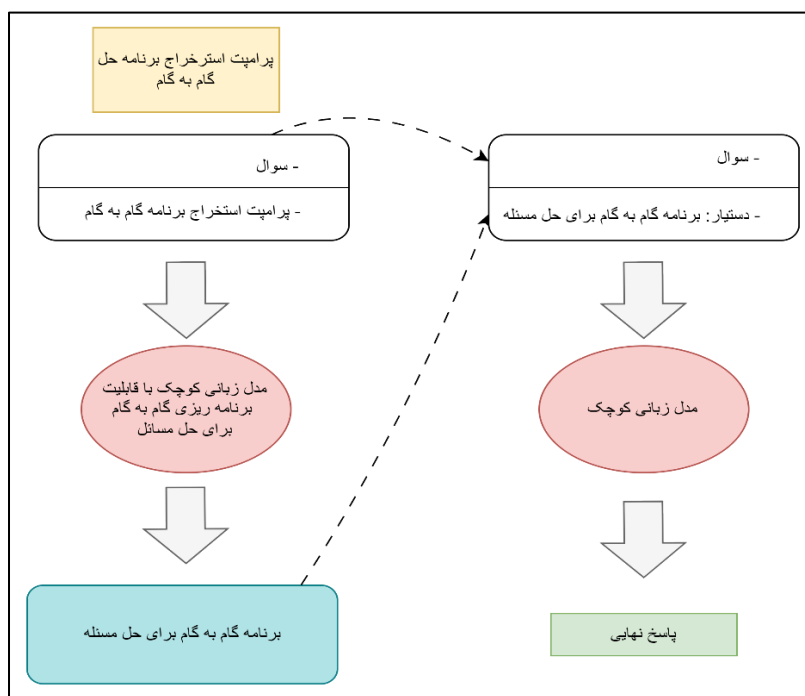
مطابق شکل ۳-۱، این معماری از یک مدل زبانی کوچک آغاز می‌شود. نخست، خروجی این مدل روی مجموعه‌ای از مسائل دامنه ریاضی بررسی می‌شود تا الگوهای تکرارشونده خطا در مرحله برنامه‌ریزی شناسایی شوند. این تحلیل خطا صرفاً با هدف گزارش ضعف‌ها انجام نمی‌شود، بلکه به‌عنوان مبنایی برای استخراج خوشه‌های مسئله به کار می‌رود؛ به این معنا که مسئله‌ها بر اساس ویژگی‌های ساختاری مشترک و نوع خطاهای مشابه گروه‌بندی می‌شوند. در طرح کلی پروژه، چهار خوشه مسئله از این مرحله استخراج می‌شوند و هر خوشه نماینده یک الگوی تکرارشونده از دشواری برای مدل است. بنابراین، خوشه‌بندی در این معماری نه یک دسته‌بندی موضوعی ساده، بلکه پلی میان تحلیل خطا و طراحی مداخله بعدی است.

پس از مشخص شدن خوشه‌ها، برای هر خوشه یک مسیر بهبود مستقل در نظر گرفته می‌شود. ایده اصلی آن است که به‌جای ساخت یک مجموعه‌داده عمومی و ناهمگن، برای هر خوشه مجموعه‌داده‌ای هدفمند ساخته شود که دقیقاً همان الگوی خطا را هدف بگیرد. در این مرحله، یک مدل زبانی بزرگ با کیفیت استدلال بالا نقش مدل معلم^{۲۴} را بر عهده می‌گیرد و برای مسائل هر خوشه، برنامه‌های حل مناسب تولید می‌کند. این برنامه‌ها به‌عنوان برچسب‌های مطلوب برای همان خوشه استفاده می‌شوند. در نتیجه، داده آموزشی برنامه‌های حل گام به گام هستند که باید مرحله نخست معماری را بهبود دهند. این بخش از معماری نشان می‌دهد که نقش مدل بزرگ در این پروژه تولید نمونه‌های راهنما برای اصلاح رفتار آن در یک مرحله مشخص است.

²⁴ Teacher model

فصل سوم: روش پیشنهادی و نتیجه‌گیری

گام بعدی در معماری، تنظیم دقیق مدل زبانی کوچک روی داده خوشه‌محور است. در این جا هدف این است که مدل در مواجهه با مسائل متعلق به یک خوشه خاص، برنامه‌های حل دقیق‌تر و انسانی‌تری تولید کند. از این‌رو، تنظیم دقیق به‌صورت هدفمند و بر مرحله برنامه‌ریزی متمرکز می‌شود. همچنین معماری به‌گونه‌ای طراحی شده است که این فرایند برای هر خوشه تکرارپذیر باشد. یعنی پس از تحلیل خطا و بهبود روی یک خوشه، همان روند در اصل بتواند برای خوشه‌های دیگر نیز به کار رود. به همین دلیل، شکل ۱-۳ یک چرخه ضمنی را نیز نشان می‌دهد: ساخت داده، تنظیم دقیق برای افزودن قابلیت تدوین برنامه گام به گام برای مسائل آن خوشه و تکرار دوباره فرایند روی همان مدل.



شکل ۲-۳ نحوه استنتاج در معماری پیشنهادی

شکل ۲-۳ لایه دوم این معماری را نشان می‌دهد و توضیح می‌دهد که در سطح تعامل با مدل، این فرایند چگونه اجرا می‌شود. ابتدا مسئله همراه با یک پرامپت ویژه به مدل داده می‌شود تا فقط برنامه حل گام‌به‌گام را استخراج کند سپس همین برنامه، همراه با خود مسئله، به مدل بازگردانده می‌شود تا مرحله حل بر پایه آن اجرا شود.

از منظر مهندسی پژوهش، این جداسازی چند مزیت مهم دارد. نخست آن‌که امکان تحلیل مستقل مرحله برنامه‌ریزی را فراهم می‌کند؛ یعنی می‌توان پیش از پرداختن به پاسخ نهایی، فقط کیفیت برنامه را بررسی کرد و خطاهای آن را دسته‌بندی نمود. دوم آن‌که تنظیم دقیق را محدود و کم‌هزینه‌تر می‌سازد، چون پروژه به‌جای دست‌کاری کل رفتار پاسخ‌گویی، فقط یک

مرحله مشخص را هدف می‌گیرد. در آخر این دکه این معماری کمک می‌کند تا خطا در حل هر مسئله تفسیرپذیر شود. زیرا اگر معلوم شود یک نوع مسئله به یک الگوی تکرارشونده از خطا منجر می‌شود، می‌توان دقیقاً همان مرحله و همان الگو را هدف قرار داد.

روند کلی معماری را می‌توان به این شکل جمع‌بندی کرد: یک مدل کوچک انتخاب می‌شود؛ خطاهای آن در مرحله برنامه‌ریزی تحلیل می‌شود؛ این خطاها به خوشه‌های مسئله تبدیل می‌شوند؛ برای یک خوشه، داده‌ای هدفمند با کمک یک مدل معلم تولید می‌شود؛ مدل کوچک روی آن داده تنظیم دقیق می‌شود؛ و در نهایت انتظار می‌رود مدل در همان خوشه، برنامه‌های حل بهتری تولید کند.

۳-۳ تحلیل خطا روی مسائل دامنه ریاضی

۳-۳-۱ تحلیل اولیه خروجی‌های مرحله برنامه‌ریزی

این کار با گردآوری خروجی‌های برنامه‌ریزی از مدل زبانی روی مجموعه‌ای از مسائل ریاضی آغاز شد و سپس این خروجی‌ها از نظر الگوهای تکرارشونده خطا تحلیل شدند. مسئله‌های انتخابی اولیه با کمک هوش مصنوعی و همچنین مسائل موجود در مجموعه داده‌های مسائل ریاضی طرح شد. برای این امر، از پرامپت‌های مختلفی استفاده شد که پس از آزمون و خطای چند پرامپت برای برنامه‌ریزی گام به گام، پرامپت پیوست انتخاب شد. هدف این نبود که قضاوت شود آیا مدل «ریاضی بلد است» یا نه، بلکه این بود که بررسی شود آیا می‌تواند به‌طور قابل اعتماد یک طرح حل درست و ساخت یافته تولید کند که به الزامات منطقی مسئله پایبند باشد. به‌ویژه وقتی صورت مسئله طولانی یا دارای مشخصات پیچیده است.

پنج الگوی خطای تکرارشونده به‌طور مداوم در خروجی‌های بررسی شده دیده شد:

۱. ناتوانی در ساختن ساختار درست وابستگی‌ها: بسیاری از مسائل به یک گراف وابستگی صریح نیاز داشتند: معرفی یک حالت نهفته میانی، شرطی‌سازی بر آن، محاسبه کمیت‌های شرطی، و سپس حاشیه‌گیری (قانون احتمال کل / قانون امید ریاضی کل). مدل اغلب این وابستگی‌ها را فرومی‌ریخت.

مثال: در مسئله نمونه‌برداری از محصول‌های کارخانه (مسئله ۱)، برنامه حل اغلب «۱۲ پیچ نمونه‌گیری شده از مخزن ترکیبی» را چنان در نظر می‌گرفت که گویی نمونه‌گیری‌های مستقلی از خط A و خط B انجام شده است. این کار متغیر تصادفی نهفته لازم را دور می‌زند: این که از ۱۲ پیچ نمونه، چند پیچ از هر خط آمده‌اند؛ متغیری که سپس توزیع مشترک تعداد پیچ‌های معیوب و تصمیم توقف خط را تعیین می‌کند.

۲. حذف قیود یا ناتوانی در ادغام آن‌ها در راه‌حل: الگوی دوم، استفاده ناقص از قیود بود. مدل گاهی قیود را ذکر می‌کرد اما آن‌ها را در عمل وارد حل نمی‌کرد، یا کاملاً نادیده می‌گرفت؛ در نتیجه عملاً نسخه‌ای ساده‌شده از مسئله را حل می‌کرد.

مثال: در مسئله شمارش حساب پیمان‌های^{۲۵} (مسئله ۲)، چندین طرح حل بر ترکیب هم‌نهشتی‌ها (استدلالی از نوع قضیه باقی‌مانده چینی) تمرکز داشتند، اما قیدهای «دقیقاً ۷ رقمی بودن» و «بخش‌پذیری دو رقم آخر بر ۴» را در مرحله شمارش وارد نمی‌کردند. به‌طور مشابه، در مسئله زمان توقف^{۲۶} «تا کشش ششم، ۳ مهره قرمز دیده شود»، بعضی از طرح‌ها «دقیقاً ۳ قرمز در ۶ برداشت» را محاسبه می‌کردند، بدون آن که شرط کنند برداشت ششم همان سومین قرمز باشد.

۳. استفاده نادرست از گزاره‌های صوری و نبود بررسی سازگاری: برخی خطاها نه از نوع حذف، بلکه از نوع استدلال صوری نادرست بودند: به‌کارگیری قضیه اشتباه، خلط تعریف‌ها، یا ادامه دادن حل بدون بررسی این که آیا پیکربندی توصیف‌شده اصلاً امکان‌پذیر است یا نه.

مثال: در مسائل هندسه، چندین طرح حل فرض‌های بدون توجیه وارد می‌کردند (برای مثال، به‌طور ضمنی فرض می‌کردند زاویه‌ای قائمه است) یا قضایای دایره را به‌اشتباه به کار می‌بردند (مثلاً تنظیم نادرست رابطه توان نقطه). در مسئله نرخ پر و خالی شدن مخزن (مسئله ۳)، بعضی از طرح‌ها عبارت‌های مربوط به نرخ تغییر حجم را با دینامیک نرخ تغییر ارتفاع مخلوط می‌کردند، بدون آن که از طریق مساحت مقطع، تبدیل لازم را به‌صراحت انجام دهند؛ و این باعث ناسازگاری بُعدی می‌شد.

۴. برنامه‌ریزی کلی و قالبی یا توسل به جست‌وجوی کل فضای حالت: یک سبک رایج در خروجی‌ها این بود که فقط گفته می‌شد: «از روش X استفاده کن» (مانند CRT، SVD)، بدون آن که تبدیل کلیدی‌ای که این روش را قابل‌استفاده می‌کند

²⁵ Modular Arithmetic Counting

مشخص شود. در مسائل سخت‌تر، این به «همه حالت‌های ممکن را بررسی کن» تبدیل می‌شود، بی‌آن‌که راهبرد تحلیلی‌ای برای هرس کردن^{۲۶} حالت‌ها پیشنهاد شود.

مثال: در مسائل بهینه‌سازی (مثلاً بیشینه کردن احتمال برد تحت برچسب‌گذاری‌های مختلف (مسئله ۴))، طرح‌ها اغلب به‌جای پیشنهاد یک راهبرد اکسترمم‌گرا (استدلال جابه‌جایی، کران‌گذاری)، به شمارش brute-force روی می‌آوردند. در مسائل نظریه اعداد نیز گاهی پیشنهاد می‌شد اعداد اول تا یک کران دلخواه آزموده شوند، به‌جای آن‌که قیودی استخراج شود که مجموعه نامزدها را ذاتاً کوچک کند.

۵. خطاهای استدلال غیرخطی: مدل اغلب مقادیر مورد انتظار متغیرهای میانی را در کمیت‌های غیرخطی مانند احتمال‌ها پس از به‌روزرسانی تصادفی، نسبت‌ها، یا رویدادهای آستانه‌ای جایگذاری می‌کرد. این کار به‌طور ساختاری، در حالت کلی معتبر نیست.

مثال: در مسئله به‌روزرسانی ظرف مهره‌ها (مسئله ۵)، بعضی از طرح‌ها «تعداد مورد انتظار مهره‌های قرمز افزوده‌شده» را محاسبه می‌کردند و سپس از آن برای ساختن احتمال نهایی کشیدن مهره قرمز استفاده می‌کردند. استدلال درست مستلزم آن است که بر هر شاخه شرطی‌سازی شود (این‌که در مرحله اول چند مهره قرمز کشیده شده‌اند) و سپس میانگین احتمال‌های شرطی حاصل گرفته شود، نه این‌که شمارش‌های امیدی را مستقیماً در یک نسبت جایگذاری کنیم.

این الگوها با مقایسه مکرر ساختار منطقی مورد انتظار هر مسئله با آن‌چه طرح حل مدل واقعاً کدگذاری می‌کرد استخراج شدند. این الگوهای خطا محدود به یک حوزه خاص نبودند (مثلاً فقط احتمال) و ضعف‌هایی نظام‌مند در بازنمایی، ادغام قیود، و استدلال پروتکلی را نشان می‌دادند.

۳-۳-۲ خوشه‌بندی مسائل

پس از شناسایی الگوهای تکرارشونده خطا، از آن‌ها برای طراحی خوشه‌هایی از مسائل استفاده کردیم که مدل را از جنبه‌های بدون وابستگی به هم محک بزنند. هدف از این خوشه‌بندی، هدفی عملی بود: ایجاد گروه‌هایی از مسائل که عوامل

²⁶ Prune

دشواری آن‌ها با هم متفاوت باشد، تا تفاوت عملکرد میان خوشه‌ها را بتوان به بارهای شناختی/ساختاری متمایز نسبت داد، نه صرفاً به «ریاضی سخت در برابر ریاضی آسان».

طراحی خوشه‌ها مستقیماً بر اساس خطاهای مشاهده‌شده در بخش قبل انجام شد و چهار خوشه انجامید:

خوشه ۱: مسائل کلامی عددی بلند با قیود متعدد^{۲۷}

این‌ها مسائلی تک‌پاراگرافی با اعداد فراوان، بندهای متعدد، و اغلب رویه‌های چندمرحله‌ای هستند (نمونه‌گیری بدون جایگذاری، به‌روزرسانی شرطی، پویایی چندفازی، آستانه‌ها/پاداش‌ها). این مسائل به‌طور جدی موارد زیر را می‌آزمایند: ساختار وابستگی، تعریف درست رویداد، و ادغام قیود. این خوشه مستقیماً رایج‌ترین ضعف‌های مشاهده‌شده را هدف می‌گیرد: شکست در بازنمایی ساختار وابستگی، انحراف در تعریف رویداد، و خطاهای ناشی از جایگذاری امید ریاضی در موقعیت‌های غیرخطی. مثال: مسئله پیچ‌های کارخانه (مسئله ۱) (نمونه‌گیری از مخزن ترکیبی، نرخ‌های خرابی، قاعده توقف در حالت تساوی) مستلزم معرفی تقسیم نمونه بین خطوط به‌عنوان یک متغیر نهفته، مدل‌سازی توزیع مشترک تعداد معیوب‌ها، و اعمال یک قاعده تصمیم‌گیری غیرخطی است.

خوشه ۲: زبان گمراه‌کننده و ابهام در مشخصات مسئله^{۲۸}

این‌ها مسائلی کوتاه تا متوسط هستند با اعداد کم اما پیچیدگی زبانی بالا (سورهای تودرتو، عباراتی مانند «دقیقاً یکی»، «مگر اینکه»، استثناءها، و پروتکل‌های شرطی). این مسائل تحلیل نحوی دقیق و انتخاب تفسیر درست را می‌آزمایند. این خوشه خطاهایی را جدا می‌کند که در آن‌ها مدل مسئله را با تفسیر اشتباه حل می‌کند و یا به درستی روند توضیح داده شده در مسئله را متوجه نمیشود.

²⁷ Long numerical, multi-constraint word problems

²⁸ Adversarial language and specification ambiguity

مثال: در مسئله پروتکل تست پزشکی (مسئله ۶) تنها در صورت مثبت بودن تست اول، تست دوم انجام می‌شود؛ و اگر هر یک از تست‌های انجام‌شده مثبت باشد، فرد آلوده تلقی می‌شود در اصل یک مسئله‌ی مدل‌سازی درخت تصمیم است، نه یک مسئله‌ی صرفاً محاسباتی.

خوشه ۳: مسائل با ساختار نمادین و جبر مبتنی بر یک قضیه^{۲۹}

این مسائل از نظر عددی بسیار سبک‌اند و بر روابط نمادین، معادلات تابعی^{۳۰}، بازگشت‌ها، تبدیل‌ها تأکید دارند. این خوشه‌ها می‌آزمایند که آیا مدل می‌تواند نمایش مناسب را انتخاب کند (جایگذاری / فرم بسته) و یا از قضیه‌ی درست برای حل استفاده کند. این خوشه اجرای محاسبات عددی را نمی‌سجد. این خوشه بررسی می‌کند که آیا مدل می‌تواند حرکت‌های ساختاری پربازده (مثل یافتن یک قضیه یا جایگذاری مناسب) را تشخیص دهد و از سرگردانی برای رسیدن به جواب جلوگیری کند.

مثال: در مسئله ۷ دستگاه حرکتی روی (x, y, z) (جایگزینی یکی از مؤلفه‌ها با مجموع دو مؤلفه‌ی دیگر) به‌طور طبیعی با شناسایی یک یک قید ماجولار حل می‌شود، نه با جست‌جوی مستقیم فضای حالت.

خوشه ۴: استدلال هندسی یا فضایی با قيود ضمنی^{۳۱}

این‌ها مسائلی هستند که در آن‌ها نمودار یا شکل به‌صورت ضمنی فرض می‌شود اما ارائه نشده است؛ بنابراین حل‌کننده باید هندسه را از متن بازسازی کند و انتشار قيود را انجام دهد. این مسائل موارد زیر را می‌آزمایند: استفاده‌ی درست از قضایا، انتخاب ساخت کمکی مناسب، و بررسی امکان‌پذیری. الگوی خطا در مسائل هندسی متفاوت بود: فرض‌های بدون توجیه، استفاده‌ی نادرست از قضایا، و نادیده گرفتن بررسی سازگاری، حتی زمانی که پیچیدگی محاسباتی پایین بود، فراوان دیده می‌شد.

²⁹ Symbolic structure and invariant-heavy algebra

³⁰ Functional Equations

³¹ Geometric or spatial reasoning with implicit constraints

مثال: مسئله‌ی زاویه در دایره (مسئله ۸) با تانژانت/وتر شامل یک قید اضافی درباره‌ی عبور یک خط از نقطه‌ی میانی است. بسیاری از طرح‌حل‌ها به‌صورت پیش‌فرض فقط از طول وتر و شعاع استفاده می‌کردند و عملاً آن شرط هندسی اضافه را نادیده می‌گرفتند.

این خوشه‌بندی دل‌خواهی یا صرفاً موضوع‌محور نبود بلکه از نوع خطاهای دیده‌شده در خروجی‌های برنامه‌ریزی استخراج شد:

- وقتی مدل به این دلیل شکست می‌خورد که وابستگی‌های ترتیبی یا حالت‌های میانی نهفته را بازنمایی نمی‌کرد، مسئله در دسته‌ی مسائل عددی بلند و چندمرحله‌ای قرار می‌گرفت .
 - وقتی شکست مدل عمدتاً ناشی از ابهام در تفسیر یا پروتکل‌های شرطی بود، مسئله در خوشه‌ی مسائل با مشخصات ابهام‌دار قرار می‌گرفت .
 - وقتی دشواری اصلی در شناسایی ناورداها یا جایگذاری‌های مناسب بود، مسئله در خوشه‌ی جبر نمادین قضیه‌محور قرار می‌گرفت .
 - وقتی خطاها عمدتاً ناشی از استفاده‌ی نادرست از قضایا، نبود ساخت کمکی، یا فرض‌های نادرست درباره‌ی امکان‌پذیری بودند، مسئله در خوشه‌ی استدلال هندسی/فضایی قرار می‌گرفت .
- این فرایند به یک چارچوب ساخت‌یافته برای تحلیل خطا منجر شد: هر خوشه یک سازوکار متفاوت شکست را هدف می‌گیرد و در نتیجه، تشخیص روشن‌تر و بهبودهای هدفمندتر برای مدل را ممکن می‌سازد.

۳-۴ بهبود مسائل یک خوشه

پس از تحلیل خطاهای مرحله برنامه‌ریزی و دسته‌بندی مسائل، خوشه اول به‌عنوان نخستین هدف بهبود انتخاب شد. این خوشه شامل مسائل عددی بلند، چندمرحله‌ای و چندقیدی بود؛ مسائلی که در آن‌ها مدل نه صرفاً به‌دلیل دشواری محاسبه، بلکه بیشتر به‌دلیل ضعف در سازمان‌دهی مسیر حل دچار خطا می‌شد. در این دسته از مسائل، خطاهایی مانند جابه‌جایی در تعریف رخداد، نادیده گرفتن برخی قیود، متصل نکردن قیود به یکدیگر، مدل‌سازی نادرست وابستگی میان مراحل، و میان‌برهای

غلط در کمیت‌های غیرخطی بیش از سایر خوشه‌ها مشاهده شد. از آن‌جا که این خطاها هم فراوان‌تر بودند و هم مستقیماً به کیفیت مرحله برنامه‌ریزی مربوط می‌شدند، خوشه اول مناسب‌ترین نقطه برای آغاز مداخله تشخیص داده شد. ایده اصلی این بود که به‌جای تلاش برای بهبود کلی و هم‌زمان همه انواع مسائل، ابتدا یک الگوی خطای غالب و تکرارشونده هدف قرار گیرد و برای همان الگو، رفتار برنامه‌ریزی مطلوب تعریف، داده مناسب تولید، و سپس مدل کوچک در همان جهت تنظیم شود. در این بخش، ابتدا رفتار برنامه‌ریزی مطلوبی که قرار بود به مدل آموزش داده شود تعریف می‌شود، و سپس فرایند تولید مجموعه داده خوشه‌محور با استفاده از مدل معلم تشریح می‌گردد.

۳-۴-۱ تعریف رفتار برنامه‌ریزی انسان‌گونه

برای مواجهه با خطاهای خوشه اول، هدف این بود که مدل یاد بگیرد پیش از ورود به حل، مسئله را مانند یک حل‌کننده انسانی سازمان‌دهی کند. در مسائل عددی بلند و چندقیدی، یک حل‌کننده انسانی معمولاً مستقیماً وارد محاسبه نمی‌شود. او ابتدا مشخص می‌کند که دقیقاً چه کمیتی خواسته شده است، کدام مجهول‌ها واقعاً نقش دارند، چه قیدهایی مسیر حل را کنترل می‌کنند، و هر مجهول یا کمیت میانی از چه چیزهایی به دست می‌آید. اگر مسئله شاخه‌های شرطی داشته باشد، معمولاً ابتدا محل فعال‌شدن هر شاخه را تشخیص می‌دهد و سپس روابط میان مراحل را به‌صورت یک ساختار وابستگی در ذهن خود نگه می‌دارد. به‌بیان دیگر، انسان پیش از حل، یک نقشه از حل می‌سازد؛ نقشه‌ای که در آن مجهول‌ها مانند گره‌ها و وابستگی‌ها مانند یال‌ها عمل می‌کنند. هدف این پروژه در خوشه اول آن بود که مرحله برنامه‌ریزی مدل کوچک تا حد امکان به چنین رفتار انسانی نزدیک شود.

در شروع این مرحله، چند دور مهندسی پرامپت^{۳۲} انجام شد تا روشن شود چه نوع دستوردهی‌ای می‌تواند از یک مدل قوی، برنامه‌ای منظم، کم‌حاشیه و قابل‌استفاده استخراج کند. نسخه‌های ابتدایی پرامپت‌ها معمولاً با چند مشکل همراه بودند. در برخی موارد، مدل به‌جای برنامه‌ریزی، مستقیماً وارد حل می‌شد و بخشی از محاسبات یا حتی پاسخ نهایی را تولید می‌کرد. در برخی موارد دیگر، خروجی بیش از حد کلی و غیرعملیاتی بود؛ مثلاً با عباراتی از جنس تحلیل کن، بررسی کن یا محاسبه

³² Prompt Engineering

کن، بدون آن که روشن کند دقیقاً چه چیزی باید از چه چیزی به دست آید. گاهی نیز مدل تعداد زیادی کمیت غیرضروری، پارامتر فرعی یا نماد کمکی معرفی می‌کرد که مسئله را از حالت ساختاریافته به یک متن شلوغ تبدیل می‌کرد. در نسخه‌هایی دیگر، ترتیب وابستگی‌ها روشن نبود؛ یعنی معلوم نمی‌شد کدام کمیت باید پیش از کدام کمیت دیگر به دست آید. این خطاها دقیقاً همان چیزی بود که قرار بود در مرحله برنامه‌ریزی حذف شود، نه اینکه در خود داده آموزشی بازتولید گردد. به همین دلیل، پرامپت نهایی به تدریج به سمتی برده شد که خروجی را از نظر نقش، محتوا و قالب به شدت مقید کند. این پرامپت که در این گزارش از آن با عنوان پرامپت استخراج برنامه حل گام به گام یاد می‌شود، به صورت صریح مدل معلم را فقط در نقش برنامه‌ریز قرار می‌داد و هرگونه حل، محاسبه، ساده‌سازی یا ارائه نتیجه عددی را ممنوع می‌کرد. افزون بر این، هدف پرامپت نه استخراج هرآنچه درباره مسئله می‌توان گفت، بلکه استخراج فقط آن چیزهایی بود که برای رسیدن به کمیت خواسته شده لازم‌اند. این تصمیم از آن جهت مهم بود که در خوشه اول، یکی از منشأهای خطا دقیقاً شلوغی و بی‌نظمی در نمایش میانی بود؛ بنابراین خروجی مطلوب باید کمینه، هدفمند و پایین‌دست‌محور می‌بود.

ساختار خروجی پرامپت استخراج برنامه حل گام به گام در پنج بخش طراحی شد: Goal، Unknowns، Core، constraints، و Dependency skeleton. بخش Goal مدل را وادار می‌کرد کمیت نهایی خواسته شده را به صورت روشن و به زبان طبیعی بازنویسی کند تا از همان ابتدا هدف حل تثبیت شود. بخش Unknowns فقط اجازه معرفی مجهول‌های واقعی یا نتایج تصمیمی را می‌داد و مدل را از ساختن متغیرهای بی‌دلیل باز می‌داشت. بخش Core constraints برای استخراج قیودی در نظر گرفته شد که واقعاً در ادامه اثر دارند، نه همه اطلاعات مسئله. بخش Dependency skeleton مهم‌ترین جزء این طراحی بود، زیرا در آن مدل باید به صورت صریح نشان می‌داد هر کمیت میانی از چه قیدها یا مجهول‌هایی به دست می‌آید. در نهایت، بخش Plan مدل را وادار می‌کرد یک ترتیب حداقلی و اجرایی برای تولید کمیت‌های میانی و رسیدن به کمیت نهایی بنویسد، بدون آن که هیچ‌یک از این مراحل را واقعاً اجرا کند. قواعد تکمیلی پرامپت نیز دقیقاً بر مبنای خطاهای نسخه‌های اولیه تنظیم شدند. برای پیشگیری از بازتولید نمادگذاری‌های طولانی و مسئله‌زا، مقرر شد تا زمانی که یک پارامتر واقعاً در نقش کمیت میانی لازم نیست، نام گذاری مستقلی برای آن انجام نشود و فقط از برچسب‌های میانی استفاده شود. برای کاهش آلودگی خروجی به حل و جبر، استفاده از معادله و عملگر نیز کنار گذاشته شد و از مدل خواسته شد وابستگی‌ها را به زبان توصیفی بیان کند. همچنین خروجی در یک بلوک مشخص محصور شد تا تنوع قالبی کاهش یابد و داده آموزشی شکل یکنواخت‌تری پیدا کند.

فصل سوم: روش پیشنهادی و نتیجه‌گیری

در مجموع، رفتار برنامه‌ریزی انسان‌گونه در این پروژه به‌معنای پرگویی یا شبیه‌سازی آزادانه فکر انسان نبود، بلکه به‌معنای بازتولید یک عادت شناختی مشخص بود: تثبیت هدف، جداسازی مجهول‌ها از داده‌ها، استخراج قیود مؤثر، ساختن اسکلت وابستگی، و تعیین حداقل توالی لازم برای رسیدن به پاسخ. در آخر این فاز، پرامپت زیر نهایی شد.

You are a human-like math problem PLANNER ONLY. Do not solve. Do not compute/derive/simplify. No numeric results.

Task: extract ONLY what is necessary to reach the asked-for quantity <ANS>.
Omit anything not used downstream.

Output exactly one plain-text block:

<<<PLAN>>>

Goal: <ANS> = (what the problem asks, in words)

Unknowns:

- <U1>=...

- <U2>=...

(only true unknowns / decision outcomes)

Core constraints:

- <C1>: ...

- <C2>: ...

Dependency skeleton:

- <R1> from <C?> and <U?>

- <R2> from <C?> and <R1>

Plan:

- Identify the minimum set of intermediates needed for <ANS>.

- Compute ... -> <R1>.

- Compute ... -> <R2>.

- Combine intermediates to express <ANS>. (still no execution)

<<<END>>>

Rules:

- Do NOT create placeholders for fixed givens; refer to them as "given in the statement".

- Do NOT name standard sub-parameters unless they are required intermediates; use <Rk> instead.

- No equations or operator symbols; describe relations in words.

- Nothing outside the block.

شکل ۳-۳ پرامپت استخراج برنامه حل گام به گام

۳-۴-۲ تولید داده با مدل معلم

پس از تعریف رفتار مطلوب مرحله برنامه‌ریزی، گام بعدی ساخت داده‌ای بود که دقیقاً همین رفتار را به مدل کوچک منتقل کند. برای این منظور، تصمیم گرفته شد خانواده‌هایی از مسئله طراحی شوند که هر خانواده یک الگوی ساختاری تکرارپذیر از دشواری را نمایش دهد. در این مرحله، خانواده‌های مسئله مستقیماً از صفر و به صورت دستی نوشته نشدند، بلکه ابتدا مشخص شد هر خانواده باید چه ویژگی‌هایی داشته باشد، چه نوع خطاهایی را هدف بگیرد، چه قیودی را در خود جای دهد و چه الگوی وابستگی‌ای را به مدل تحمیل کند؛ سپس با استفاده از یک سامانه هوش مصنوعی و با دستوردهی دقیق، ۵۰ خانواده مسئله تولید شد. بنابراین نقش هوش مصنوعی در این بخش تولید آزادانه و بدون هدایت نبود، بلکه در چارچوبی عمل می‌کرد که از پیش برای نوع مسئله، نوع دشواری، و نوع ساختار حل طراحی شده بود.

هدف این مجموعه داده، پوشش دادن مسائل عددی بلند و چندقیدی‌ای بود که حل درست آن‌ها به ساخت صریح وابستگی‌ها، نگهداشت حالت‌های پنهان، مدیریت شاخه‌های شرطی، و اتصال قیود در چند مرحله نیاز دارد. در طراحی خانواده‌ها، به طور عمدی مسئله‌هایی هدف گرفته شدند که مدل در آن‌ها معمولاً دچار رانش در تعریف رخداد، شرطی‌سازی نادرست، اشتباه در حاشیه‌گیری، خطا در امیدریاضی غیرخطی، یا حذف و جداافتادگی قیود می‌شود. به این ترتیب، مجموعه داده فقط برای افزایش تعداد نمونه‌ها ساخته نشد، بلکه برای هدف‌گیری الگوی خاصی از شکست در مرحله برنامه‌ریزی طراحی شد.

هر یک از ۵۰ خانواده، یک قالب^{۳۳} حل تکرارپذیر را نمایندگی می‌کرد؛ برای مثال، پویایی‌های چندبخشی، فعال‌شدن رژیم‌های مبتنی بر آستانه، نمونه‌گیری چندمرحله‌ای، بهینه‌سازی‌های محدودشده، فرایندهای زمان‌گسسته با به‌روزرسانی مرحله‌ای، یا مسئله‌هایی که چند قید از زبان‌های متفاوت را باید در یک مسیر حل به هم متصل کرد. در هر خانواده، کمیت‌های قابل تغییر مانند نرخ‌ها، شمارش‌ها، آستانه‌ها، ظرفیت‌ها، هزینه‌ها یا مجموعه‌ها به صورت جای‌نگهدار تعریف شدند تا بتوان از هر خانواده چندین نمونه مختلف تولید کرد.

³³ Template

پس از تثبیت خانواده‌ها، برای هر خانواده ۱۰ نمونه تولید شد و در نتیجه اندازه نهایی مجموعه داده به ۵۰۰ مسئله رسید. برای هر خانواده، یک تولیدکننده پیاده‌سازی شد که پارامترها را نمونه‌گیری می‌کرد، متن طبیعی مسئله را می‌ساخت، و در صورت لزوم با محاسبات داخلی بررسی می‌کرد که نمونه تولیدشده معتبر باشد. در این مرحله از نمونه‌گیری ردکننده^{۳۴} استفاده شد؛ یعنی اگر نمونه‌ای به حالت‌های نامعتبر، بدیهی یا فاقد ساختار چندمرحله‌ای واقعی منجر می‌شد، کنار گذاشته می‌شد و نمونه دیگری تولید می‌شد. به این ترتیب، مجموعه داده نهایی فقط شامل مسئله‌هایی شد که هم شذنی بودند، هم از نظر ساختاری غیرساده بودند، و هم واقعاً به فرایند برنامه‌ریزی نیاز داشتند.

یکی از مهم‌ترین و وقت‌گیرترین مراحل ساخت داده، بازبینی نظام‌مند خانواده‌ها و نمونه‌های تولیدشده بود. در این بازبینی، کدهای تولیدکننده هر گروه از خانواده‌ها بررسی شدند تا خطاهایی که می‌توانستند به برچسب‌گذاری پرنویز یا یادگیری الگوهای سطحی منجر شوند حذف شوند. در این فرایند، پنج دسته خطا به‌طور مکرر مشاهده شد. دسته نخست ابهام در معنای عملیاتی مسئله بود؛ برای مثال، در برخی مسئله‌ها معلوم نبود آستانه دقیقاً در چه زمانی بررسی می‌شود یا تغییر رژیم از چه لحظه‌ای اعمال می‌گردد. برای رفع این مشکل، جملات کوتاه و حداقلی در متن مسئله گنجانده شد تا ترتیب اعمال عملیات و زمان فعال‌شدن قواعد روشن شود، بدون آن‌که راه‌حل لو برود. دسته دوم مربوط به قیودی بود که در متن وجود داشتند اما در نمونه‌گیری خاص عملاً بی‌اثر می‌شدند. این وضعیت از آن جهت خطرناک بود که اگر قیدی همیشه خودبه‌خود برقرار باشد یا هیچ‌گاه فعال نشود، مدل می‌آموزد بخشی از متن را نادیده بگیرد. برای رفع این مشکل، آزمون‌های سمت تولیدکننده افزوده شد تا فقط نمونه‌هایی پذیرفته شوند که دست‌کم یک قید واقعاً رژیم حل را تغییر دهد. در اینجا بررسی‌های الزام‌آور، مقایسه با و بدون قید، و تنظیم پارامترها در نزدیکی مرزهای تصمیم به کار گرفته شد. دسته سوم به یکتانبودن پاسخ در خانواده‌های بهینه‌سازی مربوط می‌شد. در مسئله‌هایی مانند تخصیص، زمان‌بندی یا جریان، ممکن بود چند راه‌حل بهینه هم‌ارز وجود داشته باشد. اگر برچسب آموزشی قرار است یک ساختار مشخص را بازنمایی کند، وجود چند جواب بهینه مسئله‌ساز می‌شود. از این رو، در تولید این خانواده‌ها، یا همه حالت‌ها برای اندازه‌های کوچک شمارش شدند و نمونه‌های دارای چند بهینه حذف شدند، یا قانون شکستن تساوی در متن مسئله تصریح شد. دسته چهارم به مسئله‌هایی مربوط بود که برخی

³⁴ Rejection sampling

فرض‌های آماری لازم را به‌صورت روشن بیان نمی‌کردند. در چند خانواده، کمیت خواسته‌شده بدون فرض توزیعی مشخص، به‌طور یکتا تعیین نمی‌شد. این موارد یا با افزودن فرض لازم به متن اصلاح شدند، یا هدف مسئله به‌گونه‌ای بازطراحی شد که به اطلاعات ناموجود نیاز نداشته باشد. دسته پنجم نیز ناهماهنگی میان متن داستانی مسئله و محاسبه انجام‌شده در تولیدکننده بود؛ به‌ویژه در برخی خانواده‌های هندسی، ممکن بود تولیدکننده چیزی را محاسبه کند که متن مسئله عملاً آن را تعریف نکرده باشد. در این موارد، یا متن مسئله با محاسبه هم‌راستا شد، یا تولیدکننده طوری بازطراحی شد که پیکربندی هندسی کاملاً با متن سازگار باشد. حاصل این فرایند، یک مجموعه داده ۵۰۰ مسئله‌ای بود که ۴۰ خانواده آن، یعنی ۴۰۰ نمونه، برای تنظیم دقیق در نظر گرفته شد و ۱۰ خانواده باقی‌مانده برای ارزیابی کنار گذاشته شد.

پس از نهایی‌شدن خانواده‌ها و نمونه‌ها، لازم بود برای هر مسئله یک برنامه حل معلم تولید شود تا جفت ورودی-خروجی لازم برای تنظیم دقیق ساخته شود. برای این منظور، یک خط لوله مستقل برای فراخوانی دسته‌ای مدل معلم بر پایه OpenAI API پیاده‌سازی شد.

در پیاده‌سازی انجام‌شده، برای هر مسئله یک درخواست مستقل ساخته شد و همه درخواست‌ها در قالب اجرای دسته‌ای برای مدل معلم GPT-۵.۲ ارسال شدند. هر درخواست شامل دو ورودی بود: پرامپت سامانه که رفتار برنامه‌ریز را تعریف می‌کرد، و متن خود مسئله به‌عنوان ورودی کاربر. در تنظیمات درخواست، High reasoning effort فعال شده بود تا مدل معلم برای تولید برنامه‌های منظم‌تر فرصت بیشتری داشته باشد. این خروجی مستقیماً برای ساخت داده تنظیم دقیق به‌کار رفت.

نکته مهم در این مرحله آن است که مدل معلم فقط برای تولید برچسب به کار رفت و قرار نبود جایگزین مدل کوچک شود. در واقع، نقش مدل معلم فراهم‌کردن ردپای رفتاری مطلوب برای همان خوشه‌ای بود که خطاهایش پیش‌تر شناسایی شده بود. بنابراین، مجموعه داده نهایی مجموعه‌ای از مسئله‌ها همراه با برنامه‌های حل ساختاریافته بود. از این منظر، تولید داده با مدل معلم امتداد مستقیم تعریف رفتار برنامه‌ریزی انسان‌گونه بود: ابتدا رفتار مطلوب تعریف شد، سپس برای مسئله‌های خوشه اول نمونه‌های متعددی ساخته شد، و در نهایت مدل معلم همان رفتار را به‌صورت خروجی آموزشی روی آن نمونه‌ها پیاده کرد. این همان ماده خامی بود که در مرحله بعد برای تنظیم دقیق مدل کوچک به کار گرفته شد.

۳-۴-۱ تنظیم دقیق سبک‌وزن مدل با LoRA

پس از آن که برای خوشه اول مجموعه داده‌ای هدفمند از مسئله‌ها و برنامه‌های حل معلم ساخته شد، گام بعدی آن بود که رفتار برنامه‌ریزی مطلوب به مدل کوچک منتقل شود، بی‌آن که نیاز به بازآموزی کامل همه پارامترهای مدل باشد در واقع یک نوع تنظیم دقیق با نظارت^{۳۵} انجام شد. برای این منظور از تنظیم سبک‌وزن مبتنی بر LoRA و در عمل از صورت کم‌هزینه‌تر آن یعنی QLoRA استفاده شد. انتخاب این مسیر با منطق کلی پروژه سازگار بود: از یک سو محدودیت سخت‌افزاری اجازه تنظیم کامل مدل را نمی‌داد، و از سوی دیگر هدف نیز تغییر کل رفتار مدل نبود، بلکه فقط بهبود یک مؤلفه مشخص، یعنی مرحله برنامه‌ریزی، مدنظر قرار داشت. [۱۲]، [۲۱]

ایده اصلی LoRA آن است که به جای به‌روزرسانی مستقیم ماتریس وزن اصلی یک لایه، یک اصلاح کم‌رتبه به آن افزوده شود. اگر ماتریس وزن اصلی را با W_0 نشان دهیم، در LoRA وزن مؤثر به صورت

$$W=W_0+\Delta W$$

تعریف می‌شود، که در آن

$$\Delta W=BA$$

و A و B دو ماتریس کوچک با رتبه r هستند. از آن جا که معمولاً r بسیار کوچک‌تر از ابعاد کامل لایه انتخاب می‌شود، تعداد پارامترهای آموزش پذیر به شدت کاهش می‌یابد. در بسیاری از پیاده‌سازی‌ها، این اصلاح با یک ضریب مقیاس نیز همراه می‌شود و می‌توان آن را به صورت

$$W=W_0+\alpha BA$$

نوشت؛ در این جا r ظرفیت مؤلفه کم‌رتبه را کنترل می‌کند و α شدت اثر آن را. به این ترتیب، وزن اصلی منجمد باقی می‌ماند و فقط همان مؤلفه‌های کم‌رتبه آموزش داده می‌شوند. مزیت این روش آن است که هم حافظه مورد نیاز آموزش کاهش می‌یابد و هم می‌توان چند نسخه تخصصی از یک مدل را بدون تکثیر کامل همه پارامترها نگهداری کرد [۱۲].

³⁵ Supervised Fine-tuning (SFT)

QLoRA همین ایده را یک گام جلوتر می‌برد. در این روش، مدل پایه به صورت کمی‌سازی شده^{۳۶} لود می‌شود و در عین حال، گرادینانها فقط از مسیر آداپتورهای LoRA عبور می‌کنند. در نسخه کلاسیک QLoRA، وزن‌های پایه به صورت ۴ بیتی ذخیره می‌شوند، نوع کمی‌سازی از جنس NF۴ است، و برای محاسبات رو به جلو از دقت بالاتری مانند نیمه دقت استفاده می‌شود. همچنین از کمی‌سازی مضاعف برای کاهش بیشتر حافظه و از بهینه‌سازهای سازگار با این سناریو برای کنترل جهش‌های مصرف حافظه استفاده می‌شود. اهمیت QLoRA برای پروژه حاضر در این بود که اجازه می‌داد یک مدل ۱/۷ میلیارد پارامتری روی سخت‌افزار محدود پروژه به صورت عملی تنظیم شود، بدون آن‌که از مزیت آداپتورهای کم‌رتبه بگذریم. [۲۱]

در عمل، دو دستورپخت اصلی برای تنظیم سبک‌وزن آزمایش شد. در هر دو دستورکار، پایه کار QLoRA چهار بیتی با کمی‌سازی NF۴ و محاسبات نیمه دقت^{۳۷} بود. همچنین در هر دو حالت، آداپتورها فقط روی نگاشت‌های توجه^{۳۸}، یعنی q_proj ، k_proj و v_proj و o_proj اعمال شدند. دلیل این انتخاب آن بود که این بخش‌ها مستقیماً در نحوه شکل‌گیری نمایش و دنبال کردن وابستگی‌های درون‌دنباله نقش دارند و از این رو برای بهبود رفتار برنامه‌ریزی هدف مناسبی به شمار می‌رفتند. جزئیات بیشتر پیاده‌سازی در ریپازیتوری پروژه قابل مشاهده است.

۳-۴-۲ ابزارها، محیط اجرا

در مرحله تحلیل خطا، هدف آن بود که پیش از هرگونه تنظیم دقیق، رفتار پایه مدل کوچک در محیطی محلی، کنترل‌پذیر و قابل تکرار بررسی شود. برای این منظور، مدل Qwen۳-۱.۷B در قالب GGUF و با کمی‌سازی Q۸_۰ روی یک رایانه رومیزی مبتنی بر Windows اجرا شد. انتخاب قالب GGUF از آن جهت مناسب بود که این قالب برای اجرای محلی مدل‌های کمی‌سازی شده در runtime های سبک طراحی شده است و فراداده‌های لازم برای بارگذاری مدل، توکن‌ساز و قالب گفت‌وگو را نیز همراه خود نگه می‌دارد. از نظر عملی، این انتخاب اجازه می‌داد مدل با حجم تقریبی حدود دو گیگابایت در یک محیط شخصی و بدون نیاز به زیرساخت آموزشی سنگین بارگذاری شود و به صورت مستقیم در حلقه‌های ارزیابی مورد

³⁶ Quantization

³⁷ Half-precision

³⁸ Attention Mappings

استفاده قرار گیرد. هسته اصلی این محیط بر پایه llama.cpp بنا شد. این runtime به دلیل پشتیبانی مستقیم از GGUF و امکان استفاده از backend های محلی، برای تحلیل خطا گزینه مناسبی بود. در این پروژه، گرچه در فرایند آماده‌سازی backend های محلی مختلف از جمله (Vulkan نیز از منظر امکان اجرا و سازگاری بررسی شدند، اما پیکربندی نهایی و پایدار تحلیل خطا بر پایه CUDA بود.

در فاز مربوط به تولید داده با مدل معلم پیاده‌سازی به صورت خلاصه بر پایه Python و OpenAI API انجام شد و از فراخوانی‌های دسته‌ای برای ارسال مسئله‌ها، دریافت خروجی‌های مدل معلم، و بازادغام پاسخ‌ها با شناسه هر نمونه استفاده شد. در این مرحله، داده‌ها در قالب JSONL ذخیره شدند، پرامپت سامانه به صورت مستقل نگهداری شد، و خروجی‌های مدل معلم به صورت ساخت یافته برای استفاده در مرحله تنظیم دقیق ذخیره شدند.

در نهایت برای تنظیم دقیق نیز از Python، مجموعه ابزارهای Hugging Face، شامل Transformers و Datasets، کتابخانه (PEFT) برای آداپتورهای کم‌رتبه، و bitsandbytes برای لود کمی‌سازی شده و بهینه‌سازهای سازگار با QLoRA انجام شد.

همچنین داده‌های آموزشی و اعتبارسنجی از فایل‌های JSONL خوانده می‌شدند و فرایند آموزش، ارزیابی و ذخیره بهترین آداپتور در همین پشته انجام می‌گرفت. در این بخش نیز حالت تفکر مدل غیرفعال نگه داشته شد تا آموزش فقط روی رفتار برنامه‌ریزی صریح متمرکز بماند.

۳-۶ نتایج و ارزیابی

بررسی‌های دستی برنامه‌های طرح شده نشان می‌دهد خروجی‌های مدل روی داده‌های تست و آموزش از کیفیت بالاتری برخوردار هستند. در پیوست نمونه از برنامه‌های خروجی قبل و بعد از تنظیم دقیق آمده است.

۷-۳ جمع‌بندی و کارهای آینده

در این پروژه، یک معماری مرحله‌محور برای بهبود استدلال در مدل‌های زبانی کوچک در دامنه ریاضی پیشنهاد و پیاده‌سازی شد که مبنای آن جداسازی مرحله برنامه‌ریزی از مرحله حل بود. این انتخاب از آن‌جا انجام شد که بررسی‌های مفهومی و تجربی نشان دادند بخش مهمی از خطاهای مدل در مسائل چندمرحله‌ای و چندقیدی، نه از ناتوانی صرف در محاسبه، بلکه از ضعف در ساختن مسیر حل ناشی می‌شود. بر همین اساس، ابتدا تحلیل خطا روی خروجی‌های مرحله برنامه‌ریزی انجام شد و الگوهای تکرارشونده خطا در قالب چند خوشه مسئله استخراج شدند، سپس برای یکی از این خوشه‌ها که خطاهای آن غالب‌تر و بحرانی‌تر بود، یک مجموعه داده خوشه‌محور طراحی و تولید شد و مدل کوچک با استفاده از تنظیم سبک‌وزن فقط روی همان مرحله برنامه‌ریزی بهبود داده شد. روش پیشنهادی را می‌توان به صورت یک رویکرد ترکیبی در نظر گرفت که از یک سو با تجزیه فرایند استنتاج به دو مرحله برنامه‌ریزی و حل، ساختار استدلال را در زمان استنتاج به صورت نظام‌مند افزایش می‌دهد، و از سوی دیگر با تولید داده‌های هدفمند توسط مدل معلم، نوعی مقیاس‌دهی داده^{۳۹} را برای بهبود رفتار مدل کوچک فراهم می‌سازد که در نهایت از طریق تنظیم دقیق سبک‌وزن اعمال می‌شود. نتایج این بخش نشان داد که مدل تنظیم‌شده برنامه‌های حل منظم‌تر، دقیق‌تر و قابل استفاده‌تری نسبت به مدل پایه تولید می‌کند و همین بهبود در کیفیت برنامه‌ریزی، به افزایش تعداد مسئله‌هایی منجر شد که در نهایت به درستی حل شدند. البته در این پروژه، ساخت داده و تنظیم دقیق فقط برای یک خوشه انجام شد و سایر خوشه‌های استخراج‌شده هنوز به همین صورت هدف مداخله قرار نگرفته‌اند. از این رو، مسیر طبیعی کارهای آینده آن است که همین روند برای دیگر خوشه‌های شناسایی‌شده نیز تکرار شود تا بتوان به صورت نظام‌مند سایر الگوهای خطای مدل را نیز پوشش داد. در مجموع، نتایج حاصل نشان می‌دهد که معماری پیشنهادی، ظرفیت واقعی برای بهبود استدلال در دامنه ریاضی دارد و می‌تواند به عنوان چارچوبی امیدبخش برای توسعه بیشتر در خوشه‌های دیگر و در مقیاس وسیع‌تر در نظر گرفته شود.

³⁹ Data Scaling

- [1] A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, 2017.
- [2] T. B. Brown et al., “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [3] J. Kaplan et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [4] J. Hoffmann et al., “Training Compute-Optimal Large Language Models,” *arXiv:2203.15556*, 2022.
- [5] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.
- [6] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large Language Models are Zero-Shot Reasoners,” in *Advances in Neural Information Processing Systems*, vol. 35, pp. 22199–22213, 2022.
- [7] X. Wang et al., “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” *arXiv:2203.11171*, 2022.
- [8] S. Yao et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [9] L. Wang et al., “Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 2603–2619, 2023.
- [10] H. Lightman et al., “Let’s Verify Step by Step,” *arXiv:2305.20050*, 2023.
- [11] K. Cobbe et al., “Training Verifiers to Solve Math Word Problems,” *arXiv:2110.14168*, 2021.
- [12] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [13] N. Muennighoff et al., “s1: Simple test-time scaling,” *arXiv:2501.19393*, 2025.

- [14] Y. Chow, G. Tennenholtz, I. Gur, V. Zhuang, B. Dai, S. Thiagarajan, C. Boutilier, R. Agarwal, A. Kumar, and A. Faust, “Inference-Aware Fine-Tuning for Best-of-N Sampling in Large Language Models,” *arXiv:2412.15287*, 2024.
- [15] J. Chen, Z. Xun, B. Zhou, H. Qi, H. Zhang, Q. Zhang, Y. Chen, W. Hu, Y. Qu, W. Ouyang, and S. Hu, “Do We Truly Need So Many Samples? Multi-LLM Repeated Sampling Efficiently Scales Test-Time Compute,” *arXiv:2504.00762*, 2025.
- [16] J. R. Chowdhury and C. Caragea, “Zero-Shot Verification-guided Chain of Thoughts,” *arXiv:2501.13122*, 2025.
- [17] Z. Gao, L. Chen, H. Luo, J. Zhou, and B. Dai, “One-shot Entropy Minimization,” *arXiv:2505.20282*, 2025.
- [18] G. Wang, J. Li, Y. Sun, X. Chen, C. Liu, Y. Wu, M. Lu, S. Song, and Y. Abbasi Yadkori, “Hierarchical Reasoning Model,” *arXiv:2506.21734*, 2025.
- [19] A. Yang et al., “Qwen3 Technical Report,” *arXiv:2505.09388*, 2025.
- [20] Qwen Team, “Qwen/Qwen3-1.7B Model Card,” Hugging Face, 2025.
- [21] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” *arXiv:2305.14314*, 2023.

معادل انگلیسی	واژه فارسی
Attention Mappings	نگاشت‌های توجه
Chain-of-Thought Prompting	پرامپت زدن مبتنی بر زنجیره تفکر
Context Window	پنجره زمینه
Data Scaling	مقیاس داده
Embedding	دگرنمایی
Error Analysis	تحلیل خطا
Fine-tuning	تنظیم دقیق
Inference-time Scaling	مقیاس زمان استنتاج
Intermediate Representation	نمایش میانی
Large Language Model (LLM)	مدل زبانی بزرگ
Pipeline	خطلوله
Plan-and-Solve Prompting	پرامپت زدن مبتنی بر برنامه‌ریزی و حل
Process Reward Model (PRM)	مدل پاداش فرایندی
Quantization	کمی‌سازی
Reasoning tasks	وظایف استدلال
Self-Consistency	سازگاری با خود
Small Language Model (SLM)	مدل زبانی کوچک
Supervised Fine-tuning	تنظیم دقیق با نظارت
Voting and Verification	رای‌گیری و اعتبارسنجی

پیوست

- لینک ریپازیتوری گیت‌هاب کدهای پروژه به همراه دیتاست و آداپتور QLoRA:

<https://github.com/PouriaYazdani/ReasoningInSLMsForMathDomain>

- پرامپت استفاده شده برای تحلیل خطا و استخراج خوشه‌های مسائل:

```
SYSTEM_PROMPT1_extract ="""
You are a mathematical problem STEP-BY-STEP PLANNER ONLY. You must outline actions
without doing any calculations, derivations, simplifications, or symbolic transformations.

OUTPUT CONTRACT (MANDATORY)
- Output EXACTLY ONE plan block, delimited by the following sentinels:
<<>>
Steps:
1) ...
2) ...
3) ...
- First try to identify the required knowledge in order to solve the problem by extracting the
covered subjects in the problem.
- Plain text only.
- Each step is 1–2 sentences, actionable and specific.
- Use placeholders for ALL outcomes: , , , , etc.

HARD RESTRICTIONS
- DO NOT output anything before <<>>.
- DO NOT include templates, headings, examples, or explanations (e.g., "OUTPUT
FORMAT", "CORE BEHAVIOR", "TEMPLATE", etc.).
- DO NOT emit multiple plan blocks.
- DO NOT reveal any values, counts, divisors, factors, remainders, bases, simplified forms, or
final answers.
- DO NOT show equations, arrows, or math operators: =, ≈, ≡, ⇒, →, +, −, ×, *, /, ÷, ^, %, mod,
gcd, lcm, inequalities, or parentheses used for math grouping.
- DO NOT introduce new digits or number words not verbatim in the user's problem.
Placeholders like are allowed.
- DO NOT perform or describe base conversions, long division, polynomial/algebraic
manipulation, modular reductions, factoring, listing divisors, numeric casework, or numeric
comparisons.
- DO NOT perform any computation.

PLACEHOLDER POLICY
- Whenever a step would yield a concrete result, write: "Compute ... and record it as ."
- Use a new placeholder for each distinct outcome and reuse it when referenced later.

SELF-CHECK (BEFORE SENDING)
1) The output starts with <<>>.
2) There is exactly one line "Steps:" and exactly one numbered list of items.
3) No forbidden tokens appear: =, ≈, ≡, ⇒, →, +, −, ×, *, /, ÷, ^, %, mod, gcd, lcm, math
parentheses.
4) No numbers beyond those verbatim in the user's problem (placeholders exempt).
5) No computed values are shown—only instructions and placeholders.
If any check fails, silently rewrite to comply.
"""
```

- نمونه‌هایی از خروجی مدل قبل و بعد از استنتاج روی پرامپت استخراج برنامه حل گام به گام [39] روی داده‌های خوشه اول:

U: A company chooses production level x (continuous) with revenue $R(x)=192x - 3.59x^2$ and base cost $C(x)=32x$. The cost includes a piecewise penalty: add 225 if $x<5.93$, add $3.72(x-10.42)^2$ if $x>10.42$, otherwise 0. Additionally, x must satisfy $g_1(x)=x - 13 \leq 0$ and $g_2(x)=1.43x - 61.2 \leq 0$. Find the maximizing x and the profit.

مدل تنظیم دقیق شده

```
<<<PLAN>>>
Goal: <ANS> = (the maximizing production level x and the corresponding profit)
Unknowns:
- <U1>=x (the maximizing production level)
- <U2>=profit at x (the profit at the maximizing x)
Core constraints:
- <C1>: the revenue R(x) is given as a quadratic function of x
- <C2>: the base cost C(x) is given as a linear function of x, with a piecewise
penalty added depending on x's value relative to 5.93 and 10.42
- <C3>: the production level x must satisfy the two inequality constraints g1(x)
and g2(x)
Dependency skeleton:
- <R1> from <C1>: the revenue R(x) as a function of x
- <R2> from <C2>: the base cost C(x) as a function of x, with the piecewise
penalty added
- <R3> from <C3>: the profit function P(x) as a function of x
Plan:
- Identify the minimum set of intermediates needed for <ANS>: the revenue
function, the base cost function with the piecewise penalty, and the profit function
- Compute <R1> by expanding the given quadratic revenue function
- Compute <R2> by evaluating the base cost function and adding the piecewise
penalty based on x's value relative to 5.93 and 10.42
- Compute <R3> by subtracting <R2> from <R1>
- Combine intermediates to express <ANS> as the maximizing x and the
corresponding profit value
<<<END>>>
```

مدل پایه

```
<<<PLAN>>>
Goal: <ANS> = (maximizing x and the profit)
Unknowns:
- <U1>=x
- <U2>=R(x)
- <U3>=C(x)
- <U4>=g1(x)
- <U5>=g2(x)
- <U6>=penalty(x)
Core constraints:
- <C1>: R(x) = 192x - 3.59x^2
- <C2>: C(x) = 32x
- <C3>: penalty(x) = 225 if x < 5.93
- <C4>: penalty(x) = 3.72(x - 10.42)^2 if x > 10.42
- <C5>: penalty(x) = 0 if 5.93 ≤ x ≤ 10.42
- <C6>: g1(x) = x - 13 ≤ 0
- <C7>: g2(x) = 1.43x - 61.2 ≤ 0
Dependency skeleton:
- <R1> from <C1> and <U1>
- <R2> from <C2> and <U1>
- <R3> from <C3> and <U1>
- <R4> from <C3> and <U1>
- <R5> from <C6> and <U1>
- <R6> from <C7> and <U1>
Plan:
- Identify the minimum set of intermediates needed for <ANS>.
- Compute ... -> <R1>
- Compute ... -> <R2>
- Compute ... -> <R3>
- Compute ... -> <R4>
- Compute ... -> <R5>
- Compute ... -> <R6>
- Combine intermediates to express <ANS>. (still no execution)
```

A number N has a base-5 representation with exactly 6 digits, digit sum 26, and N is divisible by 2. If no such N exists in base 5, switch to base 11 while keeping the same L and S . Find the smallest valid N .

مدل تنظیم دقیق شده

<<PLAN>>>
 Goal: <ANS> = the smallest valid N in base 11, or "no such N exists in base 11"
 Unknowns:
 - <U1> = the smallest N in base 11
 - <U2> = whether such an N exists in base 11
 Core constraints:
 - <C1>: N has exactly 6 digits in base 5
 - <C2>: the digit sum of N in base 5 is 26
 - <C3>: N is divisible by 2
 - <C4>: if no such N exists in base 5, switch to base 11 while keeping the same L and S
 Dependency skeleton:
 - <R1> from <C1>: the minimum and maximum possible values of N in base 5
 - <R2> from <C2>: the digit sum constraint in base 5
 - <R3> from <C3>: divisibility by 2 in base 5
 - <R4> from <C4>: switch to base 11 if no base-5 solution exists
 Plan:
 - Identify the minimum set of intermediates needed for <ANS>.
 - Compute <R1> to find the range of N in base 5.
 - Compute <R2> to determine if the digit sum constraint is feasible in base 5.
 - Compute <R3> to determine if divisibility by 2 is feasible in base 5.
 - Combine intermediates to express <ANS> as the smallest N in base 11 if feasible, or "no such N exists in base 11" otherwise.
 <<END>>>

مدل پایه

<<PLAN>>>
 Goal: <ANS> = smallest valid N in base 11 (if no such N exists in base 5)
 Unknowns:
 - <U1>=base-5 representation of N
 - <U2>=digit sum of N in base-5
 - <U3>=divisibility by 2 in base-5
 Core constraints:
 - <C1>: N has a base-5 representation with exactly 6 digits
 - <C2>: digit sum of N in base-5 is 26
 - <C3>: N is divisible by 2
 Dependency skeleton:
 - <R1> from <C1> and <U2>
 - <R2> from <C3> and <R1>
 Plan:
 - Identify the minimum set of intermediates needed for <ANS>
 - Compute ... -> <R1>
 - Compute ... -> <R2>
 - Combine intermediates to express <ANS>

A driver travels from S to T through a directed graph with edges: S->X(time=2, toll=10); S->B(time=4, toll=7); B->X(time=8, toll=3); X->T(time=3, toll=13); X->A(time=6, toll=4); A->T(time=5, toll=2); S->A(time=5, toll=17); B->T(time=11, toll=7). If the cumulative toll paid so far exceeds 16, all subsequent tolls are discounted by 38% (the discount starts after the edge on which the cap is first exceeded). However, if the driver enters node X before time 4, edge X->T becomes unavailable. Find the minimum possible total cost defined as $2^*(\text{total time}) + 2^*(\text{total toll})$.

مدل تنظیم دقیق شده

<<PLAN>>>
 Goal: <ANS> = the minimum total cost defined as $2^*(\text{total time}) + 2^*(\text{total toll})$ over all possible paths from S to T, considering the given edge weights, tolls, and the discount rule
 Unknowns:
 - <U1> = the minimum total cost achievable by any path from S to T
 Core constraints:
 - <C1>: The driver starts at node S and follows directed edges as described in the statement.
 - <C2>: The driver pays the toll for each edge taken, and the total toll is the sum of all tolls along the path.
 - <C3>: The driver accumulates time by summing the edge times along the path.
 - <C4>: If the cumulative toll paid so far exceeds 16, all subsequent tolls are discounted by 38% (the discount starts after the edge on which the cap is first exceeded).
 - <C5>: If the driver enters node X before time 4, edge X->T becomes unavailable.
 - <C6>: The total cost is defined as $2^*(\text{total time}) + 2^*(\text{total toll})$, and we seek the minimum total cost over all possible paths from S to T.
 Dependency skeleton:
 - <R1> from <C1>: the set of all possible paths from S to T, including all edge combinations that follow the directed graph structure.
 - <R2> from <C2>: the total toll for each path, computed by summing the tolls of all edges in the path.
 - <R3> from <C3>: the total time for each path, computed by summing the times of all edges in the path.
 - <R4> from <C4>: for each path, determine whether the cumulative toll exceeds 16 at any point, and apply the 38% discount to all subsequent tolls if the cap is first exceeded.
 - <R5> from <C5>: for each path, determine whether the driver enters node X before time 4; if so, mark edge X->T as unavailable.
 - <R6> from <C6>: for each valid path (after applying <R4> and <R5>), compute the total cost as $2^*(\text{total time}) + 2^*(\text{total toll})$, and select the minimum total cost among all valid paths.
 Plan:
 - Identify the minimum set of intermediates needed for <ANS>: the set of all valid paths from S to T, the total toll for each path, the total time for each path, and the minimum total cost achievable by any path.
 - Compute <R1> by enumerating all possible paths from S to T, following the directed graph structure.
 - For each path, compute <R2> by summing the tolls of all edges in the path.
 - For each path, compute <R3> by summing the times of all edges in the path.
 - For each path, apply <R4> to determine whether the cumulative toll exceeds 16 at any point, and apply the 38% discount to all subsequent tolls if the cap is first exceeded.
 - For each path, apply <R5> to determine whether the driver enters node X before time 4; if so, mark edge X->T as unavailable.
 - Combine intermediates to express <ANS> as the minimum total cost over all valid paths (after applying <R4> and <R5>).
 <<END>>>

مدل پایه

<<PLAN>>>
 Goal: <ANS> = minimum possible total cost defined as $2^*(\text{total time}) + 2^*(\text{total toll})$
 Unknowns:
 - <U1>=total time
 - <U2>=total toll
 Core constraints:
 - <C1>: If cumulative toll exceeds 16, subsequent tolls are discounted by 38%
 - <C2>: If driver enters node X before time 4, edge X->T becomes unavailable
 Dependency skeleton:
 - <R1> from <C1> and <U2>
 - <R2> from <C2> and <R1>
 Plan:
 - Identify the minimum set of intermediates needed for <ANS>
 - Compute ... -> <R1>
 - Compute ... -> <R2>
 - Combine intermediates to express <ANS>

A cylindrical tank of radius 0.324 is initially empty and is filled with water. For the first 34 minutes, inflow is 13.08 L/min; for the next 23 minutes it is 64.94 L/min. Outflow is 6.47 L/min throughout. At the end of each minute (after applying that minute's inflow and outflow), if the water height exceeds 0.447, a sloshing loss of 13.9% of the excess volume above height 0.447 occurs instantly. Find the final water height and the total lost volume.

مدل تنظیم دقیق شده

<<<PLAN>>>
Goal: <ANS> = the minimum total cost defined as $2^*(\text{total time}) + 2^*(\text{total toll})$ over all possible paths from S to T, considering the given edge weights, tolls, and the discount rule
Unknowns:
- <U1> = the minimum total cost achievable by any path from S to T
Core constraints:
- <C1>: The driver starts at node S and follows directed edges as described in the statement.
- <C2>: The driver pays the toll for each edge taken, and the total toll is the sum of all tolls along the path.
- <C3>: The driver accumulates time by summing the edge times along the path.
- <C4>: If the cumulative toll paid so far exceeds 16, all subsequent tolls are discounted by 38% (the discount starts after the edge on which the cap is first exceeded).
- <C5>: If the driver enters node X before time 4, edge X->T becomes unavailable.
- <C6>: The total cost is defined as $2^*(\text{total time}) + 2^*(\text{total toll})$, and we seek the minimum total cost over all possible paths from S to T.
Dependency skeleton:
- <R1> from <C1>: the set of all possible paths from S to T, including all edge combinations that follow the directed graph structure.
- <R2> from <C2>: the total toll for each path, computed by summing the tolls of all edges in the path.
- <R3> from <C3>: the total time for each path, computed by summing the times of all edges in the path.
- <R4> from <C4>: for each path, determine whether the cumulative toll exceeds 16 at any point, and apply the 38% discount to all subsequent tolls if the cap is first exceeded.
- <R5> from <C5>: for each path, determine whether the driver enters node X before time 4; if so, mark edge X->T as unavailable.
- <R6> from <C6>: for each valid path (after applying <R4> and <R5>), compute the total cost as $2^*(\text{total time}) + 2^*(\text{total toll})$, and select the minimum total cost among all valid paths.
Plan:
- Identify the minimum set of intermediates needed for <ANS>: the set of all valid paths from S to T, the total toll for each path, the total time for each path, and the minimum total cost achievable by any path.
- Compute <R1> by enumerating all possible paths from S to T, following the directed graph structure.
- For each path, compute <R2> by summing the tolls of all edges in the path.
- For each path, compute <R3> by summing the times of all edges in the path.
- For each path, apply <R4> to determine whether the cumulative toll exceeds 16 at any point, and apply the 38% discount to all subsequent tolls if the cap is first exceeded.
- For each path, apply <R5> to determine whether the driver enters node X before time 4; if so, mark edge X->T as unavailable.
- Combine intermediates to express <ANS> as the minimum total cost over all valid paths (after applying <R4> and <R5>).
<<<END>>>

مدل پایه

<<<PLAN>>>
Goal: <ANS> = (final water height and total lost volume)
Unknowns:
- <U1>=final water height
- <U2>=total lost volume
Core constraints:
- <C1>: Inflow rate = 13.08 L/min for first 34 minutes
- <C2>: Inflow rate = 64.94 L/min for next 23 minutes
- <C3>: Outflow rate = 6.47 L/min throughout
- <C4>: Sloshing loss = 13.9% of excess volume above 0.447
Dependency skeleton:
- <R1> from <C1> and <U1>
- <R2> from <C2> and <U1>
- <R3> from <C3> and <U1>
- <R4> from <C4> and <R3>
Plan:
- Compute total inflow and outflow over time
- Determine final water height based on inflow and outflow
- Calculate total lost volume based on sloshing loss

- مسائل اشاره شده در بخش استخراج خطاها در بخش ۳-۱-۳:

مسئله 1- نمونه برداری از محصول های کارخانه

A factory produces bolts on two lines. Line A makes \$420\$ bolts per hour with defect probability \$0.03\$ per bolt; Line B makes \$280\$ bolts per hour with defect probability \$0.05\$ per bolt. During a \$30\$-minute window, a quality engineer randomly samples \$12\$ bolts uniformly from all bolts produced in that window. If the sample contains at least \$3\$ defective bolts, the engineer shuts down the line that contributed the larger number of defective bolts in the sample (break ties by shutting down Line B). What is the probability that Line B is shut down?"

مسئله 2- شمارش پیمانه ای

How many integers n with $1 \leq n \leq 10^8$ satisfy all of the following:

- (i) $n \equiv 17 \pmod{48}$
- (ii) $n \equiv 5 \pmod{75}$
- (iii) n is a multiple of 9
- (iv) In decimal, n has exactly 7 digits and its last two digits form a number divisible by 4

مسئله 3- نرخ پر و خالی شدن مخزن

"A right circular cylindrical tank has radius 3.5 m and height 12 m. It is initially filled with water to a height of 9 m. Water is pumped out at a constant rate of $0.18 \text{ m}^3/\text{min}$ for 40 minutes. Immediately after, a different pump adds water at a constant rate of $0.12 \text{ m}^3/\text{min}$ while a leak simultaneously drains water at a rate proportional to the current water height: $0.015h \text{ m}^3/\text{min}$ when the height is h meters. After 60 minutes of this second phase, what is the water height in the tank (in meters)?"

مسئله 4- بیشینه کردن احتمال برد تحت برچسب گذاری

"A bag contains 10 slips labeled with integers. You are told that at least 6 slips have distinct labels, and that the sum of all 10 labels is 17 . You draw 3 slips uniformly at random without replacement. If the maximum of the three labels is strictly greater than the sum of the other two labels, you win. What is the maximum possible value of your winning probability over all labelings consistent with the information?"

مسئله 5- به روز رسانی ظرف مهره ها

"An urn initially contains 5 red, 4 blue, and 3 green balls. Three balls are drawn uniformly at random without replacement. If at least two of the three are red, then 3 blue balls are added to the urn; if exactly one is red, then 2 red and 1 green are added; otherwise, 4 red are added. After the addition, one more ball is drawn. What is the probability that this fourth ball is red?"

مسائل اشاره شده ی هر خوشه در بخش ۳-۳-۲:

مسئله 6- نمونه ای از مسائل خوشه ۲

A test for a disease returns Positive or Negative. The disease prevalence is 0.02 . The test has sensitivity 0.93 and specificity 0.96 . A clinic repeats the test independently a second time only if the first result is Positive. The clinic declares "Infected" if and only if at least one of the performed tests is Positive. What is the probability that a randomly selected person declared "Infected" is actually infected?

مسئله 7- نمونه‌ای از مسائل خوشه ۳

On a board are written three integers (x, y, z) . In one move you may replace exactly one of the numbers by the sum of the other two, so (x, y, z) can become $(y + z, y, z)$ or $(x, x + z, z)$ or $(x, y, x + y)$. Starting from $(1, 1, 2)$, is it possible to reach a state where all three numbers are multiples of 7? Justify your answer.

مسئله 8- نمونه‌ای از مسائل خوشه ۴

A circle ω has center O and radius 10. A chord AB of ω has length 12. Point P lies on the minor arc AB . The tangents to ω at A and B meet at T . Given that TP intersects segment AB at its midpoint, determine the value of $\angle AOB$ in degrees.

Reasoning in Small Language Models for Mathematical Problem Solving Domain

Abstract

Despite the remarkable progress of large language models, improving reasoning ability in small language models remains a major challenge for low-cost and locally deployable applications. This project was designed and implemented with the goal of strengthening reasoning in a small language model under practical constraints such as memory, computational budget, and personal hardware limitations. In the first step, existing concepts and methods for improving reasoning in language models, including test-time scaling, planning-based methods, self-verification, and parameter-efficient fine-tuning, were studied. Based on this review, a two-stage architecture was selected in which the model first generates a solution plan and then solves the problem based on that plan. To evaluate and improve this idea, the domain of mathematical problem solving and the Qwen3-1.7B model were selected. Error analysis was then performed on the outputs of the planning stage, and mathematical problems were clustered according to recurring error patterns. Next, the cluster of long numerical multi-constraint problems was chosen as the target of intervention because it exhibited the most dominant and critical errors. For this cluster, a human-like planning behavior was defined, emphasizing the extraction of the goal, unknowns, relevant constraints, and dependency skeleton before solving. A cluster-specific dataset was then constructed, consisting of 50 problem families and 500 instances, and a teacher model was used to generate structured solution plans for each instance. Finally, the small model was fine-tuned only on the planning stage using LoRA- and QLoRA-based parameter-efficient adaptation. The results showed that the fine-tuned model produced more coherent and accurate solution plans, and this improvement in planning quality led to a higher number of problems being solved correctly. These findings indicate that the proposed architecture has meaningful potential for improving reasoning in the mathematical domain and can be extended in future work to other extracted problem clusters.

Keywords: small language models, reasoning, plan-and-solve, mathematical problem solving, light weight supervised fine-tuning, error analysis



Shahid Beheshti University
Faculty of Computer Science and Engineering

Reasoning in Small Language Models for Mathematical Problem Solving Domain

By:
Pouria Yazdani

A THESIS SUBMITTED
FOR THE DEGREE OF
BACHELOR OF SCIENCE

Supervisor
Dr. Hamed Malek

2026 Winter

